

DGOTECH · ECOSISTEMA DEPORTIVO

SportTrack / SIGDEF · API Backend

Pack de Entrega Documental

Repositorio: SportTrack-Sigdef (API)

Stack: .NET 10 · ASP.NET Core · PostgreSQL · EF Core 10 · SignalR · JWT

Documentos incluidos: 7 · **Diagramas:** 14

Fecha de emisión: 27 de septiembre de 2026

Contenido

1. Pack de Entrega — Backend SportTrack-Sigdef

Pack de Entrega — Backend SportTrack-Sigdef

Índice

1. Ficha del producto
2. Propósito y alcance
3. Cómo leer este pack
4. Glosario
5. Nota sobre documentos históricos

Pie de página

2. 01 · Requerimientos de Usuario

01 · Requerimientos de Usuario

Índice

1. Actores y roles
2. Requerimientos funcionales (RF)
3. Requerimientos no funcionales (RNF — ISO 25010)
4. Historias de usuario (HU — Gherkin)
5. Reglas de negocio (RN)
6. Matriz de trazabilidad

Pie de página

3. 02 · Casos de Uso

02 · Casos de Uso

Índice

1. Diagrama de casos de uso
2. Actores
3. Especificación de casos de uso

Pie de página

4. 03 · Diagramas de Flujo

03 · Diagramas de Flujo

Índice

1. Pipeline HTTP y autenticación/autorización
2. Ciclo de vida evento / fase / resultado
3. Motor de progresión ICF
4. Cronometraje SignalR + outbox
5. Reseteo de regata
6. Enforcement de planes SaaS
7. Mensajería aislada por sistema de origen
8. Secuencia de login

9. Backup

Pie de página

5. 04 · Manual de Uso de la API

04 · Manual de Uso de la API

Índice

1. Convenciones
2. Autenticación
3. Consumo por caso de uso
4. Ejemplos (curl / PowerShell)
5. Swagger en desarrollo
6. Manejo de errores
7. Uso para soporte
8. FAQ

Pie de página

6. 05 · Manual Técnico

05 · Manual Técnico

Índice

1. Arquitectura
 2. Stack real
 3. Estructura de la solución
 4. Arranque y Program.cs
 5. Catálogo de controllers (rutas y roles)
 6. Servicios clave
 7. Modelo de datos
 8. Seguridad
 9. SignalR
 10. Observabilidad y auditoría
 11. Backups
 12. Docker y despliegue
 13. Deuda técnica y discrepancias conocidas
 14. Migraciones y seed
 15. Estrategia offline / outbox
 16. Testing y mantenimiento
- Pie de página

7. 06 · Manual de Instalación y Despliegue

06 · Manual de Instalación y Despliegue

Índice

1. Requisitos
 2. Obtención del código
 3. Configuración (appsettings / variables)
 4. Ejecución local
 5. Migraciones
 6. Build y publicación
 7. Docker
 8. Despliegue en Render
 9. Verificación
 10. Checklist de despliegue
 11. Troubleshooting
- Pie de página

Pack de Entrega — Backend SportTrack-Sigdef

Última actualización: 2026-09-27

Documentación de entrega del backend unificado. Este pack es la fuente de verdad *de entrega*; los documentos de `docs/` (guías, técnico, seguridad, cambios) se conservan como histórico y contexto de diseño.

Índice

#	Documento	Contenido
1	<i>01-Requerimientos-Usuario.md</i>	Actores/roles, RF, RNF (ISO 25010), historias de usuario (Gherkin), reglas de negocio y matriz de trazabilidad
2	<i>02-Casos-de-Uso.md</i>	Diagrama de casos de uso y especificación de CU-01...CU-12
3	<i>03-Diagramas-de-Flujo.md</i>	Diagramas Mermaid: pipeline, estados, motor ICF, timing, reseteo, SaaS, mensajería, login, backup
4	<i>04-Manual-de-Uso-API.md</i>	Manual del integrador/administrador de la API: auth, consumo por caso de uso, errores, FAQ y uso para soporte
5	<i>05-Manual-Tecnico.md</i>	Arquitectura, stack, controllers/servicios, modelo de datos, seguridad, SignalR, deploy, deuda técnica
6	<i>06-Manual-Instalacion-Despliegue.md</i>	Requisitos, configuración, ejecución local, migraciones, Docker y despliegue en Render

Documentación histórica y de contexto (se referencia, no se reemplaza): `docs/README.md`.

 **PDF consolidado del pack:** *Pack-Entrega-SportTrack-Sigdef-Backend.pdf* Contiene los 7 documentos en orden, con todas las tablas y los **14 diagramas renderizados**. A4, 59 páginas, con índice navegable y marcadores (bookmarks) por sección.

1. Ficha del producto

Atributo	Valor
Nombre	SportTrack-Sigdef (backend unificado)
Tipo	Aplicación web API REST + tiempo real (SignalR), multi-tenant SaaS
Dominios	SportTrack: competencias, regatas, cronometraje y resultados en vivo. SIGDEF: federación (federaciones, clubes, atletas, delegados, tutores, entrenadores, traspasos, pagos). SaaS: planes, métricas, auditoría, backups
Versión documentada	2026-09-27 (migración a .NET 10)
Stack real	.NET 10 (ASP.NET Core Web API) · PostgreSQL 14+ · EF Core (Npgsql) · SignalR · JWT (HMAC-SHA512) · Cloudinary · MercadoPago SDK
Target framework	net10.0 en los 4 proyectos
Solución	SportTrack-Sigdef.sln
Despliegue	Render (Docker, multi-stage, puerto 8080)
Repositorio	c:\Users\EZQU\source\repos\SportTrack-Sigdef

 **Discrepancia corregida:** documentos anteriores (`docs/README.md` , guías) indicaban **ASP.NET Core 8** y Swagger en el puerto **5029**. El target real es **.NET 10** y los puertos locales son <http://localhost:5012> / <https://localhost:7232>. Ver §5 y el [Manual Técnico](#).

2. Propósito y alcance

El backend unifica en un solo servicio:

- **SportTrack** — gestión de eventos/regatas, pruebas, fases, progresión ICF, cronometraje en vivo (SignalR), resultados y clasificaciones.
- **SIGDEF** — administración federativa multi-tenant: federaciones, clubes, atletas federados, tutores, delegados, entrenadores, roles, documentación y traspasos.
- **SaaS** — planes comerciales (SIGDEF / SportTrack / Pack Dúo en S/M/L), enforcement de acceso, métricas globales, auditoría y backups.

2.1 Alcance

Incluye API REST bajo `api/**`, hub SignalR `/hubs/timing`, seguridad (JWT, CORS, rate limiting, security headers), persistencia PostgreSQL con migraciones automáticas al arranque, y servicios en segundo plano (sincronización de estados de evento, snapshots de audiencia).

2.2 Límites (fuera de alcance)

- **No** incluye el frontend (SPAs en `SportTrack-Front`, `FrontSigdef`, `WebSPA-SIGDEF`).
- **No** existe CI/CD ni `docker-compose` en el repositorio; el despliegue es *push a main → auto-deploy en Render*.
- **No** hay token de cronometrista separado ni validación de firma del webhook de MercadoPago (ver deuda técnica).
- **No** se aplica aún el tope `MaxAtletas` de enforcement de planes (ver deuda técnica).

3. Cómo leer este pack

Si sos...	Empezá por
Analista / QA	01 Requerimientos y 02 Casos de uso
Integrador frontend	04 Manual de Uso API y 03 Diagramas de flujo
DevOps / SRE	06 Instalación y despliegue, 05 Manual Técnico
Desarrollador backend	05 Manual Técnico (completo)
Soporte / operación	04 \$Uso para soporte y 03 \$cronometraje

4. Glosario

Término	Definición
Evento	Competencia/regata, con una o más pruebas y fechas. Estados: <code>Programada</code> , <code>EnCurso</code> , <code>Finalizada</code>
Prueba	Definición de una competencia (bote, categoría, distancia, sexo) dentro de un evento
EventoPrueba	Instancia de una prueba dentro de un evento (la unidad sobre la que se generan fases)
Fase	Serie/heat, semifinal o final dentro de un <code>EventoPrueba</code> (ej. <code>A1</code> , <code>B2</code> , <code>Final A</code>)
ICF	Federación Internacional de Canotaje. Reglamento de progresión entre series, semifinales y finales
Progresión	Mecanismo que ordena resultados y arma la siguiente etapa según un plan (<code>A1-G2</code> , variantes 1/2)
Resultado	Registro de un participante en una fase (carril, tiempo, posición, estado)
TiempoOficial	Tiempo validado oficialmente (<code>interval</code>); tiene prioridad sobre el tiempo crudo
Outbox de timing	Cola que persiste envíos de cronometrista offline para commit posterior
Cronometrista	Rol operativo que toma tiempos en pista y los envía por SignalR/outbox
Largador	Rol que da la largada y puede solicitar reseteo de regata
Tenant	Federación (multi-tenant por <code>IdFederacion</code>); el aislamiento se aplica por federación y por sistema de origen
Sistema de origen	Cabecera <code>X-Client-App</code> (<code>sporttrack</code> <code>sigdef</code>) que aísla mensajería y logins
Agente Libre	Atleta liberada por su club, sin pertenencia activa
Traspaso	Solicitud de cambio de club de un atleta federado, con flujo de aceptación/rechazo/aprobación
Enforcement de plan	Verificación en backend de los flags del <code>PlanSaaS</code> de la federación (crear roles, Live, imágenes, etc.)
Live	Resultados públicos anónimos servidos con rate limit <code>live</code> y cache corto

5. Nota sobre documentos históricos

Los documentos en `docs/` (carpetas `guias/`, `tecnico/`, `seguridad/`, `casos-de-uso/`, `criterios/`, `cambios/`, `referencia/`) se **conservan** como registro histórico de diseño y decisiones. Algunos contienen afirmaciones ya desmentidas por el código:

Documento histórico	Afirmación	Estado real
<code>docs/README.md</code>	"ASP.NET Core 8"	Target real <code>net10.0</code>
<code>docs/README.md</code> , <code>guias/operacion-local.md</code>	Swagger <code>http://localhost:5029</code>	<code>http://localhost:5012</code> (<code>launchSettings.json</code>)
<code>docs/seguridad/Diseno-Enforcement-Planes.md</code> §5.3	<code>MaxAtletas</code> implementado	No aplicado en <code>AltaAtletaService</code> (deuda)

Documento histórico	Afirmación	Estado real
docs/seguridad/Diseno-Enforcement-Planes.md	MaxTorneosActivos operativo	Migración RemovePlanTournamentLimits lo deja en -1 (sin enforcement)
docs/seguridad/*	Cookie-first JWT "fuera de alcance"	Implementado: X-Access-Token cookie + Authorization: Bearer
scripts/backfill-auditoria-id-evento.sql	Asume columna IdEvento en Auditoria	Auditoria.IdEvento es [NotMapped] ; ver inconsistencia en AuditoriaController

Estas correcciones se detallan en el [Manual Técnico §13](#).

Pie de página

- [➡ 01 · Requerimientos de Usuario](#)
- [➡ 02 · Casos de Uso](#)
- [➡ 03 · Diagramas de Flujo](#)
- [➡ 04 · Manual de Uso de la API](#)
- [➡ 05 · Manual Técnico](#)
- [➡ 06 · Manual de Instalación y Despliegue](#)
- [⬅ Índice de documentación histórica](#)

01 · Requerimientos de Usuario

Última actualización: 2026-09-27 Estándares aplicados: ISO/IEC/IEEE 29148 (ingeniería de requerimientos), ISO/IEC 25010 (calidad de producto), Gherkin (historias de usuario).

Índice

- 1. Actores y roles
- 2. Requerimientos funcionales (RF)
- 3. Requerimientos no funcionales (RNF — ISO 25010)
- 4. Historias de usuario (HU — Gherkin)
- 5. Reglas de negocio (RN)
- 6. Matriz de trazabilidad

1. Actores y roles

1.1 Actores

Actor	Descripción	Autenticación
Visitante / público	Espectador del Live (resultados en vivo)	Anónimo
Usuario autenticado	Cualquier portador de JWT válido	JWT
Operador de competencia	Admin/SuperAdmin/jueces que operan carrera	JWT
Cronometrista	Toma y envía tiempos en pista (online/offline)	JWT (rol Cronometrista)
Administrador de federación	Gestiona clubes, atletas, traspasos, mensajería	JWT (rol Admin)
Club	Gestiona su plantel y consulta su estado	JWT (rol Club)
SuperAdmin / soporte	Administración SaaS, auditoría, backups	JWT
Procesos internos	Sincronización de estados, snapshots de audiencia, purgas de outbox	Background services
Sistemas externos	MercadoPago (webhook), Cloudinary, frontends SPAs	Webhook / CORS

1.2 Roles y políticas de autorización

Los roles viven en el claim `ClaimTypes.Role` del JWT y se agrupan en constantes de `AuthRolePolicies` (proyecto `Controladores`).

Rol	RolFederacion	En RegisterableRoles	Pertenece a CompetitionOperators	Privilegiado
SuperAdmin	✓	✗ (nunca desde cliente)	✓	✓
Admin	✓	✓	✓	✓
soporte_tecnico	✓	✓	✓	✓
Club	✓	✓	✗	✗
Largador	✓	✓	✓	✗
Cronometrista	✓	✓	✓	✗
JuezControl	✓	✓	✓	✗
ControlTecnico	✓	✓	✓	✗

Constantes de rol:

Constante	Valor literal
AuthRolePolicies.CompetitionOperators	Admin,SuperAdmin,JuezControl,Largador,Cronometrista,ControlTecnico,soporte_tecnico
AuthRolePolicies.Admins	Admin,SuperAdmin,soporte_tecnico
AuthRolePolicies.RegisterableRoles	Club,Admin,Largador,Cronometrista,JuezControl,ControlTecnico,soporte_tecnico
AuthRolePolicies.PrivilegedRoles	Admin,SuperAdmin,soporte_tecnico

Roles de juez (`PlanSaaSAccessHelper.IsJudgeRole`): `Largador`, `Cronometrista`, `JuezControl`, `ControlTecnico`.

Política por defecto: `FallbackPolicy = RequireAuthenticatedUser` — todo endpoint es privado salvo `[AllowAnonymous]` explícito (Live, login, health, webhook, frontend-error).

2. Requerimientos funcionales (RF)

2.1 Autenticación y seguridad

ID	Requerimiento	Endpoint(s)	Roles
RF-01	Iniciar sesión con usuario/contraseña, emitiendo JWT (HMAC-SHA512) y cookie X-Access-Token ; lee X-Client-App para aislar por sistema de origen	POST api/Auth/login	Anónimo (rate auth)
RF-02	Cerrar sesión y limpiar la cookie de acceso	POST api/Auth/logout	Anónimo
RF-03	Solicitar reseteo de contraseña con respuesta genérica (no filtra existencia)	POST api/Auth/solicitar-reset-password	Anónimo (rate auth)
RF-04	Registrar usuarios, validando rol contra CanCreateRole del plan	POST api/Auth/register	Admins (rate auth)
RF-05	Listar usuarios del ámbito (federación/sistema)	GET api/Auth/usuarios	Autenticado
RF-06	Cambiar contraseña de usuario / actualizar perfil	PUT api/Auth/usuarios/{id}/password PUT api/Auth/usuarios/{id}/perfil	Autenticado
RF-07	Activar/desactivar usuario	PATCH api/Auth/usuarios/{id}/toggle-activo	Admin, SuperAdmin, soporte_tecnico
RF-08	Eliminar usuario (baja de login)	DELETE api/Auth/usuarios/{id}	Admin, SuperAdmin, soporte_tecnico
RF-09	Obtener el perfil del usuario autenticado (con enforcement de plan/estado)	GET api/Auth/me	Autenticado

2.2 Eventos

ID	Requerimiento	Endpoint(s)	Roles
RF-10	Listar eventos y obtener detalle/fases	GET api/Eventos , GET api/Eventos/{id} , GET api/Eventos/{id}/fases	Auth / Live anónimo
RF-11	Listar próximos eventos	GET api/Eventos/proximos	Anónimo
RF-12	Crear, editar y eliminar eventos; consultar debug	POST api/Eventos , PUT api/Eventos/{id} , DELETE api/Eventos/{id} , GET api/Eventos/debug	Autenticado
RF-13	Gestionar pruebas del evento y largada de maratón	POST api/Eventos/{id}/pruebas , PUT api/Eventos/pruebas/{id} , DELETE api/Eventos/pruebas/{id} , POST api/Eventos/{id}/pruebas/largada	Autenticado

2.3 Fases y progresión ICF

ID	Requerimiento	Endpoint(s)	Roles
RF-14	Generar fases automáticamente por plan ICF (A1-G2), con cabezas de serie y pre-generación de SF/finales	POST api/Fases/Generar/{eventoPruebaId}	CompetitionOperators
RF-15	Generar fases manualmente	POST api/Fases/GenerarManual/{eventoPruebaId}	CompetitionOperators
RF-16	Generar largada única de maratón (una fase Largada , carril = número de competidor)	POST api/Fases/GenerarLargadaMaraton	CompetitionOperators
RF-17	Promover etapa (valida resultados completos y arma la siguiente ronda)	POST api/Fases/Promover/{eventoPruebaId}	CompetitionOperators
RF-18	Iniciar / finalizar / reiniciar fase y enviar a revisión	POST api/Fases/{id}/Iniciar , .../Finalizar , .../Reiniciar , .../EnviarARevision	CompetitionOperators
RF-19	Actualización masiva de fases y consulta de auditoría de progresión	POST api/Fases/BatchUpdate , GET api/Fases/ProgresionAudit/{id}	CompetitionOperators / Live
RF-20	Eliminar fase	DELETE api/Fases/{id}	CompetitionOperators

2.4 Resultados y Live

ID	Requerimiento	Endpoint(s)	Roles
RF-21	Consultar resultados de una fase (cache 30 s) con posiciones y estados	GET <code>api/Resultados/Fase/{faseId}</code>	Live anónimo (rate live)
RF-22	Guardar resultados en lote (limpieza de tiempo/posición en DNS/DNF/DSQ; audita SAVE_TIMING; emite ResultadoActualizado)	PUT <code>api/Resultados/BatchUpdate</code>	CompetitionOperators

2.5 Inscripciones

ID	Requerimiento	Endpoint(s)	Roles
RF-23	CRUD de inscripciones + filtros por evento/prueba/club y toggle de seeding	GET <code>api/Inscripciones</code> , GET <code>/id</code> , GET <code>/registro</code> , GET <code>/evento-prueba/{id}</code> , GET <code>/evento/{id}/club/{id}</code> , POST, PUT <code>/id</code> , DELETE <code>/id</code> , PATCH <code>/id/toggle-seeding</code>	Autenticado

2.6 Catálogos y participantes

ID	Requerimiento	Endpoint(s)	Roles
RF-24	CRUD de participantes y filtro por club	<code>api/Participantes</code> , GET <code>api/Participantes/club/{clubId}</code>	Autenticado
RF-25	CRUD de clubes SportTrack, botes, categorías y distancias	<code>api/Clubes</code> , <code>api/Botes</code> , <code>api/Categorias</code> , <code>api/Distancias</code>	Autenticado
RF-26	Operaciones legacy de pruebas por evento	<code>api/legacy/eventos/{idEvento}/pruebas</code>	Autenticado

2.7 SIGDEF (federación)

ID	Requerimiento	Endpoint(s)	Roles
RF-27	CRUD de atletas federados + alta completa + liberar a Agente Libre	<code>api/Atleta</code> (GET paged, POST, POST full, POST <code>/id/liberar</code>)	Autenticado
RF-28	CRUD de clubes, delegados, entrenadores, tutores, personas, roles y asociaciones atleta-tutor	<code>api/Club</code> , <code>api/DelegadoClub</code> , <code>api/Entrenador</code> , <code>api/Tutor</code> , <code>api/Persona</code> , <code>api/Rol</code> , <code>api/AtletaTutor</code>	Autenticado
RF-29	Gestionar trasposos: periodos, búsqueda, validaciones, aceptación/rechazo por origen, aprobación forzada, cancelación, auditoría y exportación CSV	<code>api/Traspaso/**</code>	Autenticado
RF-30	Subir documentación a Cloudinary (multipart) y listar/eliminar	POST <code>api/Documentacion/upload</code> , GET <code>api/Documentacion/persona/{id}</code> , DELETE <code>api/Documentacion/{id}</code>	Autenticado
RF-31	CRUD de usuarios SIGDEF	<code>api/Usuario</code>	Autenticado

2.8 Pagos y MercadoPago

ID	Requerimiento	Endpoint(s)	Roles
RF-32	Consultar historial de pagos y registrar pago	GET <code>api/Pagos/historial</code> , POST <code>api/Pagos/registrar</code>	Autenticado
RF-33	Togglear/solicitar pago de clubes, togglear pago de atletas e inscripciones	PUT <code>api/Pagos/clubes/{id}/toggle</code> , <code>.../solicitar-pago</code> , PUT <code>api/Pagos/atletas/{id}/toggle</code> , PUT <code>api/Pagos/inscripciones/{id}/toggle</code>	Autenticado
RF-34	Eliminar pago (individual o masivo)	DELETE <code>api/Pagos/{id:int}</code> , DELETE <code>api/Pagos/bulk</code>	Autenticado
RF-35	Crear preferencia de pago MercadoPago y CRUD de transacciones	POST <code>api/PagoTransaccion/preferencia</code> + CRUD	Autenticado
RF-36	Recibir webhook de MercadoPago	POST <code>api/Notificacion/webhook</code>	Anónimo ⚠ sin validación de firma (deuda)

2.9 Mensajería

ID	Requerimiento	Endpoint(s)	Roles
RF-37	Listar/leer hilos, responder, marcar leído y contar no leídos, todo aislado por X-Client-App	<code>api/mensajes/**</code>	SuperAdmin, Admin, Club

ID	Requerimiento	Endpoint(s)	Roles
RF-38	Enviar mensaje masivo y consultar campañas	POST <code>api/mensajes/hilos/masivo</code> , GET <code>api/mensajes/campanas</code> , GET <code>api/mensajes/campanas/{id}</code>	SuperAdmin, Admin

2.10 SaaS

ID	Requerimiento	Endpoint(s)	Roles
RF-39	Consultar planes (público) y depurar el plan propio	GET <code>api/SaaS/planes</code> , GET <code>api/SaaS/debug-me</code>	Anónimo / SuperAdmin, soporte
RF-40	Actualizar plan (calcula <code>PrecioAnual</code> ; <code>MaxTorneosActivos</code> = -1)	PUT <code>api/SaaS/planes/{id}</code>	SuperAdmin
RF-41	Asignar plan a club, listar estado de clubes y toggle de activación	POST <code>api/SaaS/asignar-plan</code> , GET <code>api/SaaS/clubes-status</code> , PATCH <code>api/SaaS/clubes/{id}/toggle-activo</code>	SuperAdmin, Admin, soporte
RF-42	Crear federación con admin (mapea error único 23505)	POST <code>api/SaaS/create-federacion</code>	SuperAdmin
RF-43	Consultar métricas globales	GET <code>api/SaaS/global-metrics</code>	SuperAdmin, soporte

2.11 Auditoría y soporte

ID	Requerimiento	Endpoint(s)	Roles
RF-44	Consultar auditoría general y agrupada por eventos; registrar acción de cliente; purgar por evento o por id	<code>api/Auditoria/**</code>	Autenticado
RF-45	Diagnóstico: chequeo de eventos y búsqueda	GET <code>api/Diagnostic/check-eventos</code> , GET <code>api/Diagnostic/search/{query}</code>	Admin, SuperAdmin, soporte
RF-46	Soporte: eventos, acciones de cliente, timing-outbox, logs y reporte de error de frontend	<code>api/Support/**</code>	Autenticado / Anónimo (<code>frontend-error</code>)

2.12 Backups

ID	Requerimiento	Endpoint(s)	Roles
RF-47	Descargar backup (<code>full</code> con <code>pg_dump</code> / <code>federacion</code> con INSERTs) y consultar historial	GET <code>api/Backup/download?</code> <code>scope=full federacion&idFederacion=N</code> , GET <code>api/Backup/history</code>	SuperAdmin, soporte

2.13 Timing Outbox (cronometraje offline)

ID	Requerimiento	Endpoint(s)	Roles
RF-48	Encolar, listar pendientes, commit por fase, flush y eliminar envíos de timing	POST <code>api/timing-outbox</code> , GET <code>/pending</code> , POST <code>/flush</code> , POST <code>/{{faseId}}/commit</code> , DELETE <code>/{{faseId}}</code>	CompetitionOperators
RF-49	Commit de cola desde soporte	POST <code>api/Support/timing-outbox/{{faseId}}/commit</code> , DELETE <code>api/Support/timing-outbox/{id}</code>	Autenticado

2.14 Audience (audiencia en vivo)

ID	Requerimiento	Endpoint(s)	Roles
RF-50	Consultar audiencia live, picos y capacidad, y ajustar capacidad	GET <code>api/Audience/live</code> , <code>/peaks</code> , <code>/capacity</code> , PUT <code>api/Audience/capacity</code>	SuperAdmin, soporte

2.15 Notificaciones en tiempo real

ID	Requerimiento	Mecanismo	Roles
RF-51	Notificar mensajes nuevos al usuario (<code>newMessageReceived</code> → grupo <code>user_{username}</code>)	SignalR	Usuario destino
RF-52	Notificar eventos nuevos a la federación (<code>newEventCreated</code> → grupo <code>fed_{federacionId}</code>)	SignalR	Miembros de federación
RF-53	Notificar cambios de estado de resultados y solicitudes de cambio de pago	SignalR	Operadores

2.16 Health

ID	Requerimiento	Endpoint(s)	Roles
RF-54	Verificar salud del servicio y de la base de datos	GET <code>api/Health</code> , GET <code>api/Health/db</code>	Anónimo

3. Requerimientos no funcionales (RNF — ISO 25010)

3.1 Adecuación funcional

ID	Requerimiento
RNF-01	El backend debe cubrir los dominios SportTrack, SIGDEF y SaaS en un único despliegue y una única base de datos.
RNF-02	Toda operación de escritura relevante debe quedar auditada (acción, usuario, detalle, timestamp UTC).

3.2 Eficiencia de desempeño

ID	Requerimiento
RNF-03	Las lecturas Live deben usar cache en memoria (<code>ILiveCacheService</code>) con TTL de 30–45 s. La fase cachea 30 s.
RNF-04	La base de datos debe usar <code>EnableRetryOnFailure(3)</code> y <code>CommandTimeout = 30 s</code> .
RNF-05	El rate limiting debe limitar <code>auth</code> a 20 req/min/IP y <code>live</code> a 120 req/min/IP (HTTP 429).

3.3 Compatibilidad

ID	Requerimiento
RNF-06	CORS por lista blanca con <code>AllowCredentials</code> (no <code>AllowAnyOrigin</code>), incluyendo dominios propios, Vercel, Capacitor/Ionic y localhost de desarrollo.
RNF-07	Compatibilidad con señalización SignalR via query string <code>?access_token</code> y cookie <code>X-Access-Token</code> .
RNF-08	Debe soportar clientes móviles empaquetados con Capacitor (<code>capacitor://localhost</code> , <code>ionic://localhost</code>).

3.4 Usabilidad

ID	Requerimiento
RNF-09	Swagger UI disponible solo en Development, con botón Authorize Bearer.
RNF-10	Los errores deben devolver mensajes sanitizados y códigos HTTP coherentes (400/401/403/404/429/500).

3.5 Fiabilidad

ID	Requerimiento
RNF-11	Las migraciones EF Core deben aplicarse automáticamente al arranque.
RNF-12	La sincronización de estados de evento corre como background service cada 15 min (nunca retrocede de estado).
RNF-13	El outbox de timing debe reintentar/purgar entradas con TTL de 24 h.

3.6 Seguridad

ID	Requerimiento
RNF-14	Autenticación JWT firmada con HMAC-SHA512; <code>TokenKey</code> obligatorio fuera de Development.
RNF-15	Autorización deny-by-default (<code>FallbackPolicy</code>) + roles por endpoint.
RNF-16	Lockout: 5 intentos fallidos → <code>EstaActivo = false</code> .
RNF-17	Security headers: <code>X-Content-Type-Options</code> , <code>X-Frame-Options: DENY</code> , <code>Referrer-Policy</code> , <code>Permissions-Policy</code> .
RNF-18	Reglas de carga de archivos: máx. 6 MB; <code>jpg/jpeg/png/webp/gif/pdf</code> .
RNF-19	Contraseñas almacenadas con BCrypt.
RNF-20	Recomendación: agregar validación de firma al webhook de MercadoPago (deuda actual).

3.7 Mantenibilidad

ID	Requerimiento
RNF-21	Arquitectura en 4 proyectos con separación por capas (Api / Controladores / AccesoDatos / Entidades).
RNF-22	Codificación UTF-8 forzada por <code>Directory.Build.props</code> .
RNF-23	Desacoplamiento vía interfaces (Services/Repositories) e inyección de dependencias.

3.8 Portabilidad

ID	Requerimiento
RNF-24	Target <code>net10.0</code> ; contenedor Docker <code>mcr.microsoft.com/dotnet/aspnet:10.0-noble</code> con <code>EXPOSE 8080</code> .
RNF-25	Debe leer la cadena de conexión de <code>ConnectionStrings:DefaultConnection</code> o de la variable <code>DATABASE_URL</code> (normalizando <code>postgres://</code>).

3.9 Capacidad y escalabilidad (SaaS multi-tenant)

ID	Requerimiento
RNF-26	Aislamiento multi-tenant por federación y por sistema de origen (<code>X-Client-App</code>).
RNF-27	La capacidad de audiencia (<code>AudienceMonitoring:SoftCapacity</code>) debe ser configurable y observable (picos guardados).

4. Historias de usuario (HU — Gherkin)

HU-01 — Inicio de sesión (RF-01, RN-01, RN-04)

Como usuario registrado, **quiero** iniciar sesión **para** acceder a mi panel según mi rol y sistema.

Feature: Inicio de sesión

 Scenario: Login exitoso

 Given un usuario activo con credenciales válidas

 And la cabecera `X-Client-App` indica un sistema con acceso habilitado

 When envío POST `/api/Auth/Login` con `username` y `password`

 Then recibo HTTP 200 con el JWT y la cookie `X-Access-Token`

 And el token contiene el `claim` de rol

 Scenario: Bloqueo por intentos fallidos

 Given un usuario con 4 intentos fallidos previos

 When envío un quinto login con contraseña incorrecta

 Then el usuario queda con `EstaActivo = false`

 And recibo HTTP 401

 Scenario: Exceso de intentos por IP

 Given una IP que superó 20 requests de auth en un minuto

 When envío POST `/api/Auth/Login`

 Then recibo HTTP 429 Too Many Requests

HU-02 — Alta de federación con administrador (RF-42, RN-03)

Como SuperAdmin, **quiero** crear una federación con su admin **para** habilitar un nuevo tenant.

Feature: Alta de federación

 Scenario: Creación exitosa

 Given un SuperAdmin autenticado

 And un nombre de federación no existente

 When envío POST `/api/SaaS/create-federacion` con datos de federación y admin

 Then recibo HTTP 200/201 con la federación creada

 And existe un usuario Admin asociado a la federación

 Scenario: Federación duplicada

 Given una federación ya existente con el mismo identificador único

 When envío POST `/api/SaaS/create-federacion`

 Then recibo un error de conflicto mapeado desde el código PostgreSQL 23505

HU-03 — Alta completa de atleta federado (RF-27, RN-05)

Como Admin de federación, **quiero** dar de alta un atleta con sus tutores **para** federarlo.

Feature: Alta de atleta

Scenario: Alta completa

Given un Admin autenticado

And un documento normalizado no registrado

When envío POST /api/Atleta/full con datos del atleta y tutores

Then el atleta queda asociado a la federación del admin

And se crean/actualizan las entidades Persona/Tutor correspondientes

Scenario: Documento duplicado

Given un atleta ya registrado con el mismo documento

When envío POST /api/Atleta/full con ese documento

Then el sistema resuelve el registro existente (upsert) sin duplicar

 **Deuda:** el tope MaxAtletas del plan **no** se aplica todavía en AltaAtletaService.

HU-04 — Generación automática de fases ICF (RF-14, RN-01)

Como operador de competencia, **quiero** generar las series automáticamente **para** programar la regata.

Feature: Generación de fases

Scenario: Generación por plan automático

Given un EventoPrueba con N participantes confirmados

And un plan de progresión resoluble para N

When envío POST /api/Fases/Generar/{eventoPruebaId}

Then se crean las fases con carriles asignados en prioridad [5,4,6,3,7,2,8,1,9]

And los cabezas de serie se distribuyen sin carriles duplicados

And se registra auditoría GENERATE_HEATS_AUTO

Scenario: Cantidad inválida

Given un EventoPrueba sin participantes suficientes

When envío POST /api/Fases/Generar/{eventoPruebaId}

Then recibo HTTP 400 con mensaje descriptivo

HU-05 — Promoción de etapa (RF-17, RN-01)

Como operador, **quiero** promover los clasificados **para** armar semifinales/finales.

Feature: Promoción de etapa

Scenario: Promoción con resultados completos

Given todas las fases de la ronda con resultados completos

When envío POST /api/Fases/Promover/{eventoPruebaId}

Then se asignan los clasificados a la siguiente ronda por mejor tiempo

And se registra auditoría PROMOTE_STAGE

Scenario: Resultados incompletos

Given una fase sin todos sus resultados oficiales

When envío POST /api/Fases/Promover/{eventoPruebaId}

Then recibo HTTP 400 y no se modifica ninguna fase

HU-06 — Cronometraje offline con outbox (RF-48, RN-06)

Como cronometrista, **quiero** registrar tiempos sin conexión **para** no perder datos en pista.

Feature: Outbox de timing

Scenario: Envío encolado

Given un cronometrista autenticado sin conectividad al hub
 When encolo tiempos vía POST /api/timing-outbox
 Then cada envío queda persistido con estado pendiente y TTL de 24 h
 And el upsert se realiza por (FaseId, Username)

Scenario: Commit de la cola

Given una fase con envíos pendientes y la carrera finalizada
 When envío POST /api/timing-outbox/{faseId}/commit
 Then se aplica BatchUpdate sobre los resultados
 And se ejecuta FinalizarFase o EnviarARevision según corresponda
 And los envíos quedan marcados como procesados

HU-07 — Reseteo de regata (RF-18, RN-07)

Como largador, **quiero** reiniciar una regata tras una mala largada **para** repetirla limpiamente.

Feature: Reseteo de regata

Scenario: Reseteo con motivo

Given una fase en curso
 When solicito el reseteo con motivo de al menos 5 caracteres
 Then se limpian los tiempos registrados
 And la fase vuelve al estado Programada
 And se emite el evento RaceReset y se audita RESET_RACE

Scenario: Motivo insuficiente

Given una fase en curso
 When solicito reseteo con un motivo de menos de 5 caracteres
 Then recibo HTTP 400 y la fase no cambia

HU-08 — Aprobación de traspaso (RF-29, RN-08)

Como Admin de federación, **quiero** aprobar un traspaso **para** mover un atleta de club.

Feature: Traspaso de atleta

Scenario: Flujo completo

Given una solicitud de traspaso con aceptación del club de origen
 When envío POST /api/Traspaso/{id}/aprobar
 Then el atleta cambia de club
 And se registra la auditoría del traspaso

Scenario: Rechazo del origen

Given una solicitud de traspaso pendiente
 When el club de origen envía POST /api/Traspaso/{id}/rechazar-origen
 Then la solicitud queda rechazada y el atleta no cambia de club

HU-09 — Mensajería aislada por sistema (RF-37, RN-02)

Como Admin, **quiero** mensajear a mis clubes sin cruzar información **para** respetar el aislamiento SIGDEF/SportTrack.

Feature: Mensajería aislada

Scenario: Aislamiento por X-Client-App

Given un Admin en el sistema sigdef
 When consulto GET /api/mensajes/hilos con X-Client-App: sigdef
 Then solo veo hilos de mi federación y sistema de origen
 And no veo hilos de SportTrack ni de otras federaciones

Scenario: Permiso de escritura

Given un Admin y un Club de la misma federación
 When el Admin responde un hilo del Club
 Then el mensaje se persiste y se notifica al usuario destino

HU-10 — Backup de base de datos (RF-47)

Como SuperAdmin, **quiero** descargar un backup **para** respaldar la información.

Feature: Backup

Scenario: Backup completo

Given un SuperAdmin autenticado

When envío GET /api/Backup/download?scope=full

Then recibo un archivo generado con pg_dump (PGPASSWORD por variable de entorno)

And la operación queda en el historial

Scenario: Backup por federación

Given un SuperAdmin autenticado y un idFederacion válido

When envío GET /api/Backup/download?scope=federacion&idFederacion=N

Then recibo un archivo con INSERTs de esa federación

HU-11 — Live público de resultados (RF-21, RNF-03)

Como espectador, **quiero** ver resultados en vivo sin login **para** seguir la regata.

Feature: Live público

Scenario: Consulta anónima con cache

Given una fase en curso

When envío GET /api/Resultados/Fase/{faseId} sin token

Then recibo HTTP 200 con los resultados (servidos desde cache si está vigente)

Scenario: Límite de tráfico Live

Given una IP que superó 120 requests live en un minuto

When consulto un endpoint Live

Then recibo HTTP 429 Too Many Requests

5. Reglas de negocio (RN)

ID	Regla	Implementación
RN-01	Progresión ICF: planes A1–G2 con SlotRule / BtSlotRule ; alias PlanX ; validación de carriles duplicados; ResolveDefaultPlan por cantidad (10-18→A, 19-27→B, ..., 64-72→G, variante 1/2). Los rankings excluyen Descalificado / DNS / DNF y prefieren TiempoOficial . Assign lanza excepción si el carril está ocupado.	Fase/Progression/ProgressionPlanRegistry , ProgressionEngine
RN-02	Aislamiento por sistema de origen: X-Client-App (sporttrack / sigdef) filtra hilos, campañas y no-leídos. PuedeEscribir permite SuperAdmin↔Admin y Admin↔Club de la misma federación; masivo solo SuperAdmin/Admin.	MensajeService , MensajeriaSistemaOrigen
RN-03	Multi-tenant: toda consulta/escritura de federación se acota por IdFederacion (tenant).	TenantScopeHelper , TenantProvider
RN-04	Enforcement de planes: RegisterAsync valida CanCreateRole (Club ← AccesoDashboardClub ; jueces ← AccesoControlesLive). IDs 1-3 SIGDEF, 4-6 SportTrack, 7-9 Pack Dúo; AccesoControlesLive solo ids 6 y 9. LoginAsync / GetMeAsync bloquean usuarios inactivos/bloqueados por pago/vencidos o con X-Client-App sin acceso.	PlanSaaSAccessHelper , AuthService
RN-05	Alta de atleta: NormalizarDocumento + BuscarPorDocumentoAsync (3 estrategias) + UpsertParticipanteAsync + EnsureAtletaFederacionAsync para evitar duplicados. Topo MaxAtletas NO aplicado (deuda).	AltaAtletaService
RN-06	TTL outbox 24 h: upsert por (FaseId, Username) ; CommitAsync aplica BatchUpdate + FinalizarFase / EnviarARevision ; FlushPendingAsync y PurgeExpiredAsync . No existe token de cronometrista separado; su sesión dura 24 h vs. 5 h para otros roles.	TimingOutboxService , SessionLifetimePolicy
RN-07	Reseteo de regata: motivo ≥ 5 caracteres; limpia tiempos; vuelve a Programada ; audita RESET_RACE ; emite RaceReset .	FaseService.ReiniciarFaseAsync , TimingHub.RequestResetRace
RN-08	Traspasos: estados y transiciones con aceptación/rechazo del origen, aprobación forzada, rechazo, cancelación y auditoría.	TraspasoService
RN-09	Lockout: 5 intentos fallidos → EstaActivo = false .	AuthService
RN-10	Estados de evento: Programada → EnCurso → Finalizado por día local; nunca retrocede; sincronizado por background service cada 15 min y al arranque.	EventoEstadoSyncService

ID	Regla	Implementación
RN-11	Asignación de carriles: prioridad [5,4,6,3,7,2,8,1,9] ; cabezas de serie distribuidos; carril único por fase.	FaseService.GenerarFasesAutoAsync
RN-12	Maratón: una única fase Largada con Carril = NumeroCompetidor ; audita GENERATE_LARGADA_MARATON .	FaseService.GenerarLargadaMaratonAsync
RN-13	Resultados DNS/DNF/DSQ: se limpian tiempo y posición; audita SAVE_TIMING ; emite ResultadoActualizado .	ResultadoBatchUpdateService
RN-14	Precio anual de plan: PrecioAnual = Precio × 12 × (1 - Descuento/100) , redondeo AwayFromZero . MaxTorneosActivos = -1 (sin límite efectivo).	SaaSService.UpdatePlanAsync
RN-15	Auditoría de errores: excepciones inesperadas se loguean y auditan como ERROR_FATAL y devuelven mensaje sanitizado.	ExceptionMiddleware

6. Matriz de trazabilidad

HU	RF asociados	RN	CU (ver 02)	Endpoints principales
HU-01	RF-01, RF-09	RN-04, RN-09	CU-01	POST api/Auth/login , GET api/Auth/me
HU-02	RF-42	RN-03, RN-04	CU-02	POST api/SaaS/create-federacion
HU-03	RF-27	RN-05	CU-03	POST api/Atleta/full
HU-04	RF-14, RF-15	RN-01, RN-11	CU-04	POST api/Fases/Generar/{id}
HU-05	RF-17	RN-01	CU-05	POST api/Fases/Promover/{id}
HU-06	RF-48, RF-49	RN-06	CU-07, CU-08	api/timing-outbox/** , api/Support/timing-outbox/**
HU-07	RF-18	RN-07	CU-06	POST api/Fases/{id}/Reiniciar
HU-08	RF-29	RN-08	CU-10	api/Traspaso/**
HU-09	RF-37, RF-38	RN-02	CU-11	api/mensajes/**
HU-10	RF-47	—	CU-12	api/Backup/**
HU-11	RF-21, RF-10, RF-13	RNF-03, RNF-05	CU-01 (Live)	GET api/Resultados/Fase/{id}

RNF	Dónde se verifica
RNF-03	ResultadosController (cache 30 s), LiveCacheService
RNF-04	Program.cs (AddDbContext)
RNF-05	Program.cs (RateLimiter auth / live)
RNF-11	Program.cs (migraciones al arranque)
RNF-12	EventoEstadoBackgroundService
RNF-14...RNF-19	Program.cs , AuthService , SecurityHeadersMiddleware , FileUploadRules

Pie de página

- 02 · Casos de Uso
- 03 · Diagramas de Flujo
- 04 · Manual de Uso de la API
- 05 · Manual Técnico
- Volver al índice del Pack

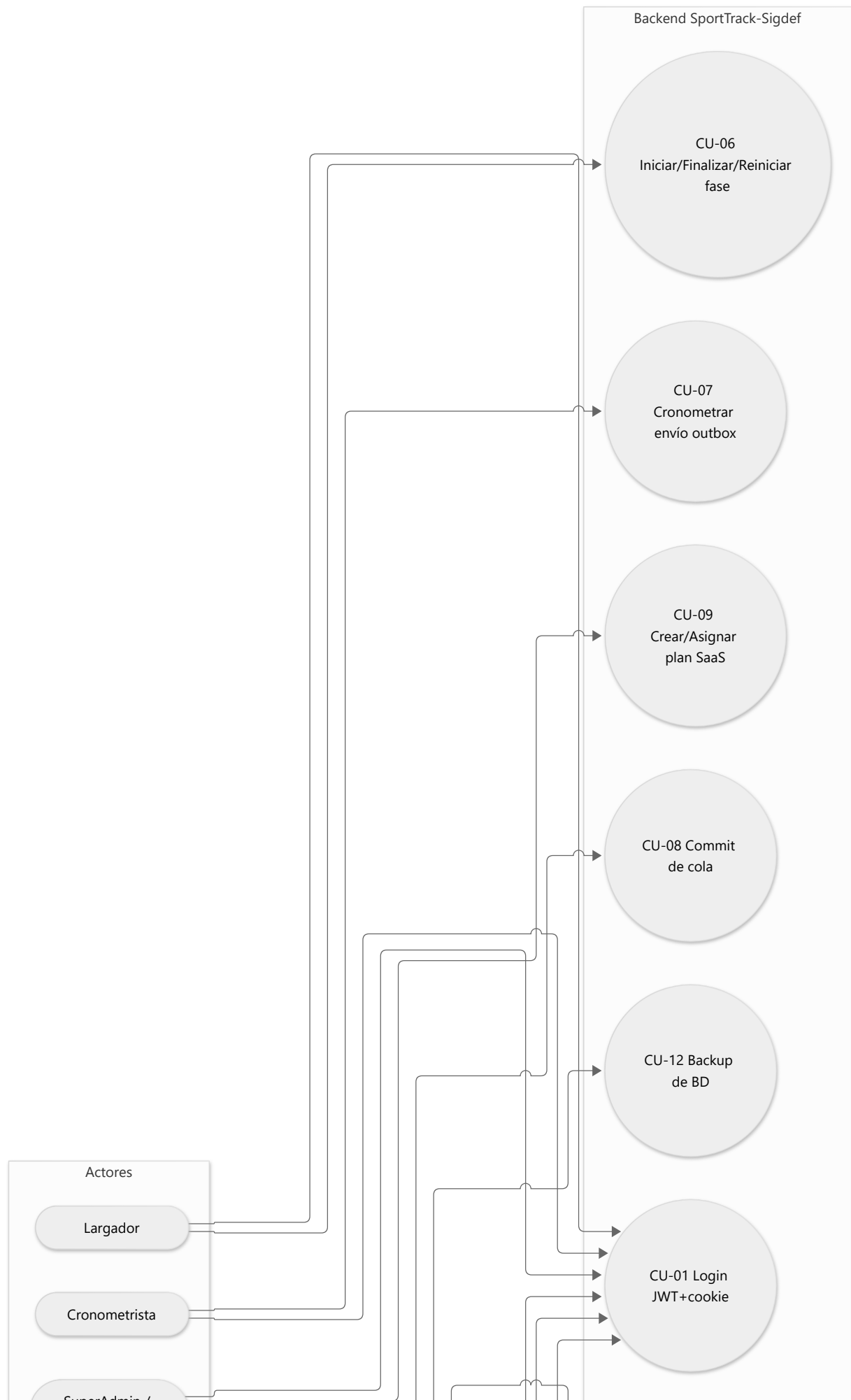
02 · Casos de Uso

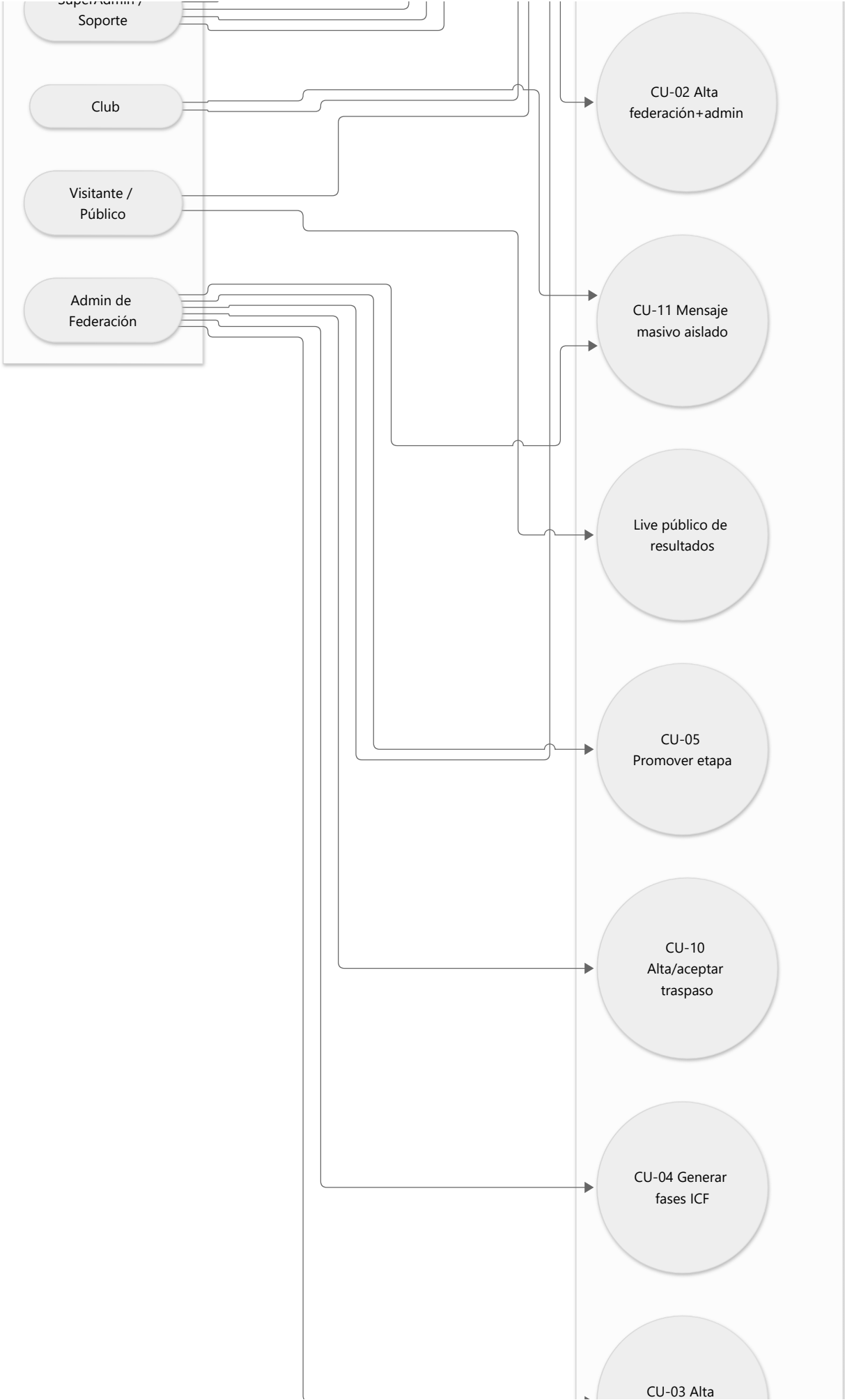
Última actualización: 2026-09-27 Notación: UML (casos de uso), IDs CU-## .

Índice

- 1. Diagrama de casos de uso
 - 2. Actores
 - 3. Especificación de casos de uso
 - CU-01 · Login y emisión de JWT + cookie
 - CU-02 · Alta de federación con administrador (SaaS)
 - CU-03 · Alta completa de atleta
 - CU-04 · Generar fases automáticamente
 - CU-05 · Promover etapa ICF
 - CU-06 · Iniciar / finalizar / reiniciar fase
 - CU-07 · Cronometrar y enviar tiempos (outbox)
 - CU-08 · Commit de cola de timing (soporte)
 - CU-09 · Crear / asignar plan SaaS
 - CU-10 · Alta y aceptación de traspaso
 - CU-11 · Mensaje masivo aislado
 - CU-12 · Backup de base de datos
-

1. Diagrama de casos de uso







2. Actores

Actor	Casos de uso principales
Visitante/Público	CU-01, Live
Cronometrista	CU-01, CU-07
Largador	CU-01, CU-06
Admin de Federación	CU-01, CU-03, CU-04, CU-05, CU-06, CU-10, CU-11
Club	CU-01, CU-11
SuperAdmin / Soporte	CU-01, CU-02, CU-08, CU-09, CU-12

3. Especificación de casos de uso

CU-01 · Login y emisión de JWT + cookie

Campo	Valor
ID	CU-01
Actores	Todos los usuarios registrados
RF / RN	RF-01, RF-09 · RN-04, RN-09
Precondición	El usuario existe, está <code>EstaActivo</code> y no bloqueado por pago/vencimiento
Postcondición	JWT emitido y devuelto (body + cookie <code>X-Access-Token</code>)

Flujo principal

- El actor envía `POST api/Auth/login` con `username / password` (rate limit `auth` , 20/min/IP) y cabecera `X-Client-App` .
- `AuthService` valida credenciales (BCrypt) y el estado del usuario (activo, plan al día, sistema de origen con acceso).
- `TokenService` emite JWT firmado con HMAC-SHA512.
- El controlador setea la cookie `X-Access-Token` y devuelve `200` con el token.
- El cliente usa `Authorization: Bearer` (preferido) o la cookie.

Flujos alternativos

- **A1 — Credenciales inválidas:** incrementa el contador de intentos; a los 5 fallos `EstaActivo = false` ; devuelve `401` .
- **A2 — Sin acceso por sistema de origen:** `X-Client-App` sin flag `Acceso*` → `401/403` .
- **A3 — Bloqueado por pago o vencido:** `403` .
- **A4 — Exceso de intentos por IP:** `429` .

Reglas: cookie-first implementado (`OnMessageReceived` prioriza `Authorization` → `?access_token` → cookie `X-Access-Token`).

CU-02 · Alta de federación con administrador (SaaS)

Campo	Valor
ID	CU-02
Actor	SuperAdmin
RF / RN	RF-42 · RN-03, RN-04
Precondición	SuperAdmin autenticado
Postcondición	Federación creada con un usuario Admin asociado

Flujo principal

- `POST api/SaaS/create-federacion` con datos de federación y admin.
- `SaaSService.CreateFederacionWithAdminAsync` crea la federación y el usuario Admin.
- Se devuelve la federación creada.

Flujos alternativos

- **A1 — Duplicado:** la base devuelve `23505` (unique violation) y el servicio lo mapea a un error de conflicto legible.

CU-03 · Alta completa de atleta

Campo	Valor
ID	CU-03
Actor	Admin de Federación
RF / RN	RF-27 · RN-05
Precondición	Admin autenticado con federación resuelta
Postcondición	Atleta federado creado/actualizado, con tutores asociados

Flujo principal

1. `POST api/Atleta/full` con datos del atleta (documento, categoría, club) y tutores.
2. `AltaAtletaService` normaliza el documento y busca existencia (3 estrategias).
3. Si no existe, hace `UpsertParticipanteAsync`; luego `EnsureAtletaFederacionAsync` para la federación resuelta.
4. Se asocian los tutores (`AtletaTutor`).

Flujos alternativos

- **A1 — Documento duplicado:** se reutiliza/actualiza el registro existente.
- **A2 — Federación no resoluble:** `ResolverFederacionIdAsync` falla → `400`.

 **Deuda:** el tope `MaxAtletas` del plan **no** se valida aún.

CU-04 · Generar fases automáticamente

Campo	Valor
ID	CU-04
Actor	Admin / Operador de competencia
RF / RN	RF-14 · RN-01, RN-11
Precondición	<code>EventoPrueba</code> con participantes confirmados; plan ICF resoluble
Postcondición	Fases creadas con carriles y cabezas de serie asignados

Flujo principal

1. `POST api/Fases/Generar/{eventoPruebaId}` (rol `CompetitionOperators`).
2. `ProgressionPlanRegistry.ResolveDefaultPlan` elige plan por cantidad (10-18→A ... 64-72→G, variante 1/2).
3. `FaseService.GenerarFasesAutoAsync` crea 9 carriles con prioridad `[5,4,6,3,7,2,8,1,9]`, distribuye cabezas de serie y **pre-genera** SF/finales.
4. Valida carriles duplicados; audita `GENERATE_HEATS_AUTO`.

Flujos alternativos

- **A1 — Carril ocupado:** `ProgressionEngine.Assign` lanza excepción → `400`.
- **A2 — Cantidad no soportada:** plan no resoluble → `400`.

CU-05 · Promover etapa ICF

Campo	Valor
ID	CU-05
Actor	Admin / Operador
RF / RN	RF-17 · RN-01
Precondición	Resultados completos de la ronda actual
Postcondición	Clasificados asignados a la siguiente ronda (A/B/C por mejor tiempo)

Flujo principal

1. `POST api/Fases/Promover/{eventoPruebaId}`.
2. `FaseService.PromoverFasesAsync` valida resultados completos, calcula rankings (excluye `Descalificado` / `DNS` / `DNF`, prefiere `TiempoOficial`) y asigna clasificados.
3. Audita `PROMOTE_STAGE`.

Flujos alternativos

- **A1 — Resultados incompletos:** 400 , sin cambios.

CU-06 · Iniciar / finalizar / reiniciar fase

Campo	Valor
ID	CU-06
Actor	Largador / Operador
RF / RN	RF-18 · RN-07
Precondición	Fase en estado compatible
Postcondición	Fase en EnCurso / Finalizada / Programada (reseteo)

Flujo principal

1. **Iniciar:** POST api/Fases/{id}/Iniciar → fase EnCurso ; emite RaceStarted / globalRaceStarted .
2. **Finalizar:** POST api/Fases/{id}/Finalizar → posición + oficial; emite RaceFinished / globalRaceOfficialized .
3. **Reiniciar:** POST api/Fases/{id}/Reiniciar con motivo (≥5 caracteres) → limpia tiempos, vuelve a Programada , audita RESET_RACE , emite RaceReset .
4. **Enviar a revisión:** POST api/Fases/{id}/EnviarARevision → emite RaceInReview .

Flujos alternativos

- **A1 — Motivo de reseteo insuficiente:** 400 .

La vía SignalR equivalente es RequestStartRace / RequestResetRace (hub /hubs/timing) .

CU-07 · Cronometrar y enviar tiempos (outbox)

Campo	Valor
ID	CU-07
Actor	Cronometrista
RF / RN	RF-48 · RN-06
Precondición	Cronometrista autenticado (sesión 24 h)
Postcondición	Tiempos encolados o transmitidos en vivo

Flujo principal (online)

1. Se une a race_{faseId} (JoinRaceGroup) y consulta hora con GetServerTime .
2. Envía SendTime (o RecordLap) → el hub retransmite TimeReceived / globalTimeReceived .
3. UpdateResultStatus actualiza el estado del resultado.

Flujo principal (offline)

1. Sin conexión, encola cada tiempo con POST api/timing-outbox (upsert por (FaseId, Username) , TTL 24 h).
2. GET api/timing-outbox/pending lista pendientes; POST api/timing-outbox/flush intenta vaciado.

Flujos alternativos

- **A1 — Entradas expiradas:** PurgeExpiredAsync las elimina tras 24 h.

No existe token de cronometrista separado; se usa el JWT estándar con SessionLifetimePolicy extendida.

CU-08 · Commit de cola de timing (soporte)

Campo	Valor
ID	CU-08
Actor	SuperAdmin / Soporte
RF / RN	RF-48, RF-49 · RN-06
Precondición	Fase con envíos pendientes y carrera finalizada/lista
Postcondición	Resultados aplicados y fase finalizada o en revisión

Flujo principal

- 1. `POST api/Support/timing-outbox/{faseId}/commit` (o `POST api/timing-outbox/{faseId}/commit`).
- 2. `TimingOutboxService.CommitAsync` aplica `BatchUpdate` de resultados.
- 3. Ejecuta `FinalizarFase` o `EnviarARevision` según el caso.
- 4. Marca los envíos como procesados; `DELETE api/Support/timing-outbox/{id}` permite descartar entradas.

Flujos alternativos

- **A1 — Sin pendientes:** operación no-op con mensaje informativo.

CU-09 · Crear / asignar plan SaaS

Campo	Valor
ID	CU-09
Actor	SuperAdmin
RF / RN	RF-39, RF-40, RF-41 · RN-04, RN-14
Precondición	SuperAdmin autenticado
Postcondición	Plan actualizado y/o asignado a club/federación

Flujo principal

- 1. `GET api/SaaS/planes` (público) lista planes con flags derivados.
- 2. `PUT api/SaaS/planes/{id}` actualiza precios; $\text{PrecioAnual} = \text{Precio} \times 12 \times (1 - \text{Descuento}/100)$; $\text{MaxTorneosActivos} = -1$.
- 3. `POST api/SaaS/asignar-plan` asigna plan a club.
- 4. `GET api/SaaS/clubes-status` muestra uso de atletas vs `MaxAtletas` y `PlanALDia`.

Flujos alternativos

- **A1 — Rol no permitido por plan:** el alta de usuario falla vía `CanCreateRole`.

CU-10 · Alta y aceptación de traspaso

Campo	Valor
ID	CU-10
Actor	Admin / Club de origen
RF / RN	RF-29 · RN-08
Precondición	Periodo de traspaso activo; atleta federado
Postcondición	Solicitud creada, aceptada/rechazada por origen y aprobada/rechazada

Flujo principal

- 1. `GET api/Traspaso/periodo-activo` verifica periodo; `GET api/Traspaso/buscar-atletas` busca candidatos.
- 2. `POST api/Traspaso` crea la solicitud.
- 3. `GET api/Traspaso/{id}/validaciones` muestra reglas aplicables.
- 4. `POST api/Traspaso/{id}/aceptar-origen` o `.../rechazar-origen`.
- 5. `POST api/Traspaso/{id}/aprobar` (o `.../forzar`) o `.../rechazar`; `.../cancelar`.
- 6. `GET api/Traspaso/auditoria` y `GET api/Traspaso/export/csv` para trazabilidad.

CU-11 · Mensaje masivo aislado

Campo	Valor
ID	CU-11
Actor	SuperAdmin / Admin
RF / RN	RF-37, RF-38 · RN-02
Precondición	Actor con rol habilitado para masivo
Postcondición	Campaña creada y mensajes entregados/notificados

Flujo principal

- 1. `POST api/mensajes/hilos/masivo` con contenido y destinatarios (solo SuperAdmin/Admin).
- 2. `MensajeService` filtra destinatarios por federación y sistema de origen (`X-Client-App`).
- 3. Se crea `CampanaEnvio` y se notifica por SignalR (`newMessageReceived` → `user_{username}`).
- 4. `GET api/mensajes/campanas` y `.../campanas/{id}` consultan el resultado.

Flujos alternativos

- **A1 — Rol no autorizado:** 403 .
- **A2 — Intento de cruzar sistemas:** el filtro de origen excluye destinatarios fuera del sistema.

CU-12 · Backup de base de datos

Campo	Valor
ID	CU-12
Actor	SuperAdmin / Soporte
RF / RN	RF-47
Precondición	SuperAdmin autenticado; pg_dump disponible (≥ versión del server)
Postcondición	Archivo de backup descargado e historial actualizado

Flujo principal

1. GET api/Backup/download?scope=full → BackupService ejecuta pg_dump con PGPASSWORD por variable de entorno.
2. GET api/Backup/download?scope=federacion&idFederacion=N → genera INSERTs de esa federación.
3. GET api/Backup/history lee el historial desde Auditoría.

Flujos alternativos

- **A1 — pg_dump ausente o mismatch de versión:** error controlado; revisar contenedor (postgresql-client-18).

Pie de página

- ➡ 03 · Diagramas de Flujo
- ➡ 04 · Manual de Uso de la API
- ➡ 01 · Requerimientos de Usuario · [Volver al índice del Pack](#)

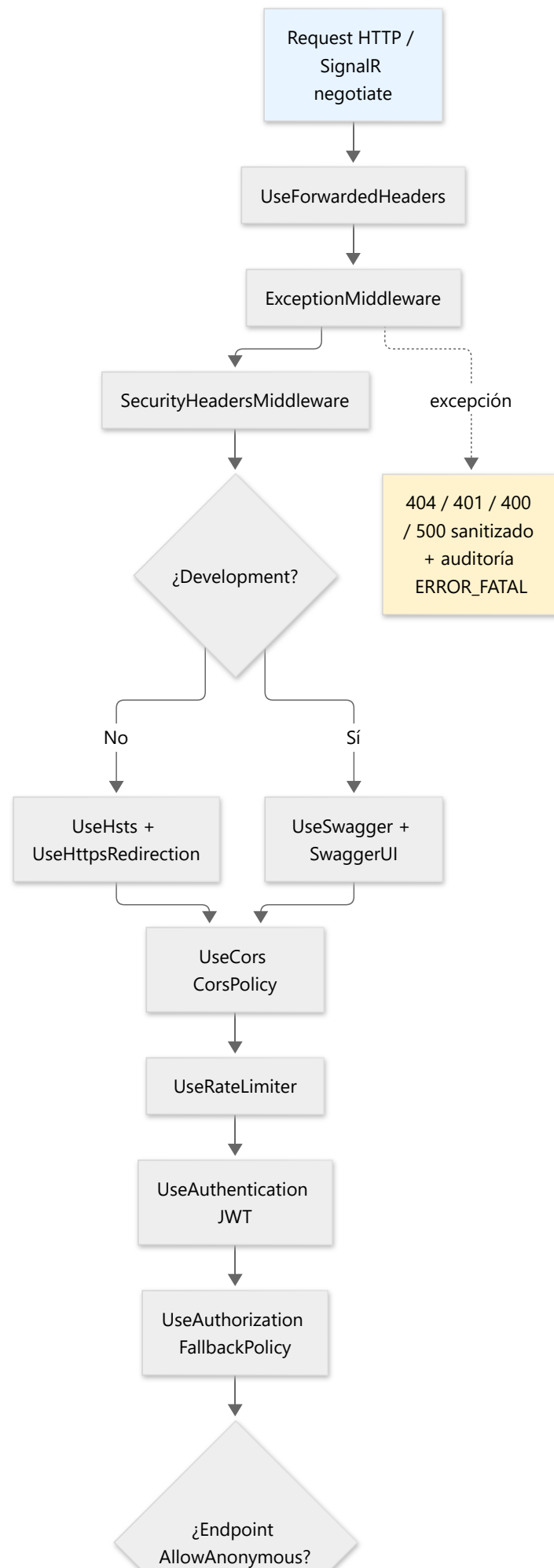
03 · Diagramas de Flujo

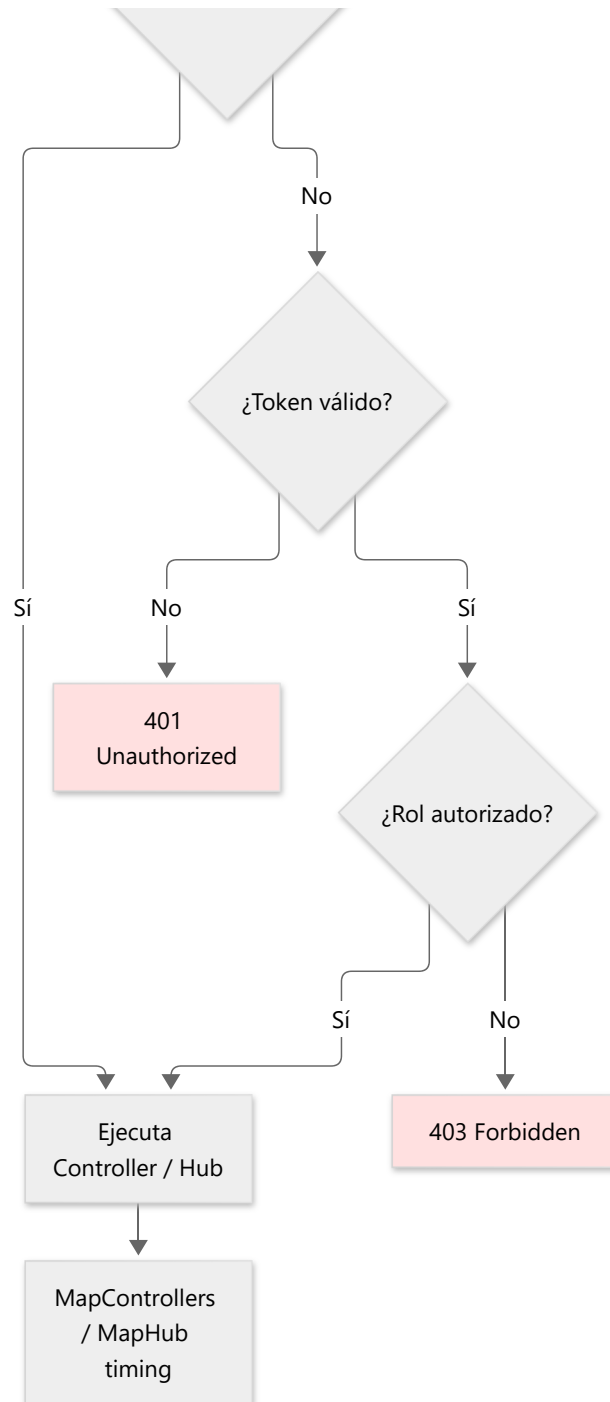
Última actualización: 2026-09-27 Notación: Mermaid (flowchart, stateDiagram-v2, sequenceDiagram). Todos los diagramas verificados contra el código.

Índice

- 1. Pipeline HTTP y autenticación/autorización
 - 2. Ciclo de vida evento / fase / resultado
 - 3. Motor de progresión ICF
 - 4. Cronometraje SignalR + outbox
 - 5. Reseteo de regata
 - 6. Enforcement de planes SaaS
 - 7. Mensajería aislada por sistema de origen
 - 8. Secuencia de login
 - 9. Backup
-

1. Pipeline HTTP y autenticación/autorización



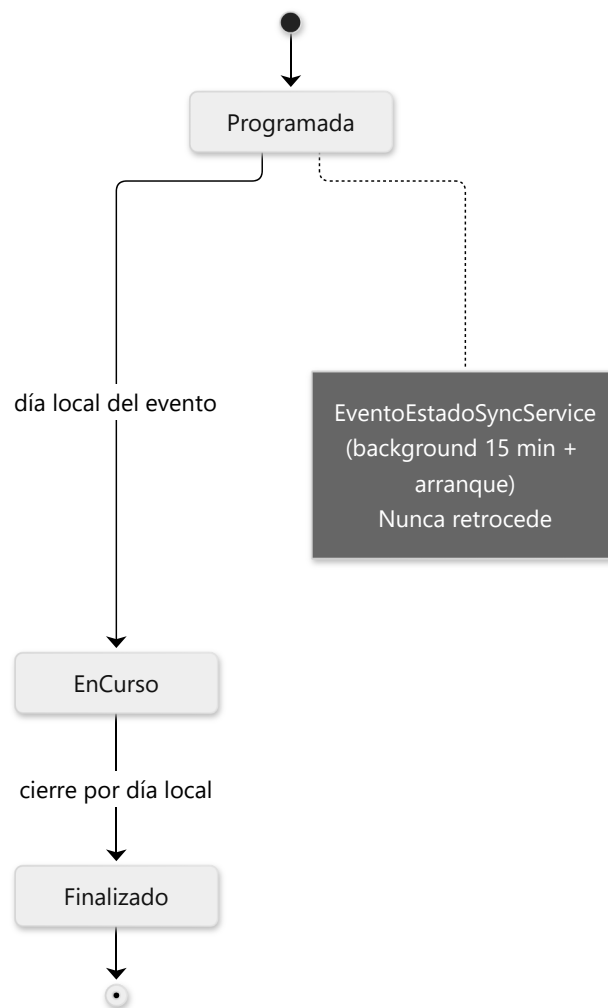


Orden real del pipeline (Program.cs): `ExceptionHandlerMiddleware` → `SecurityHeadersMiddleware` → `HSTS/HTTPS` (no Dev) → `Swagger` (Dev) → `CORS` → `RateLimiter` → `Authentication` → `Authorization` → `MapControllers` → `MapHub<TimingHub>` (`('/hubs/timing').AllowAnonymous()`).

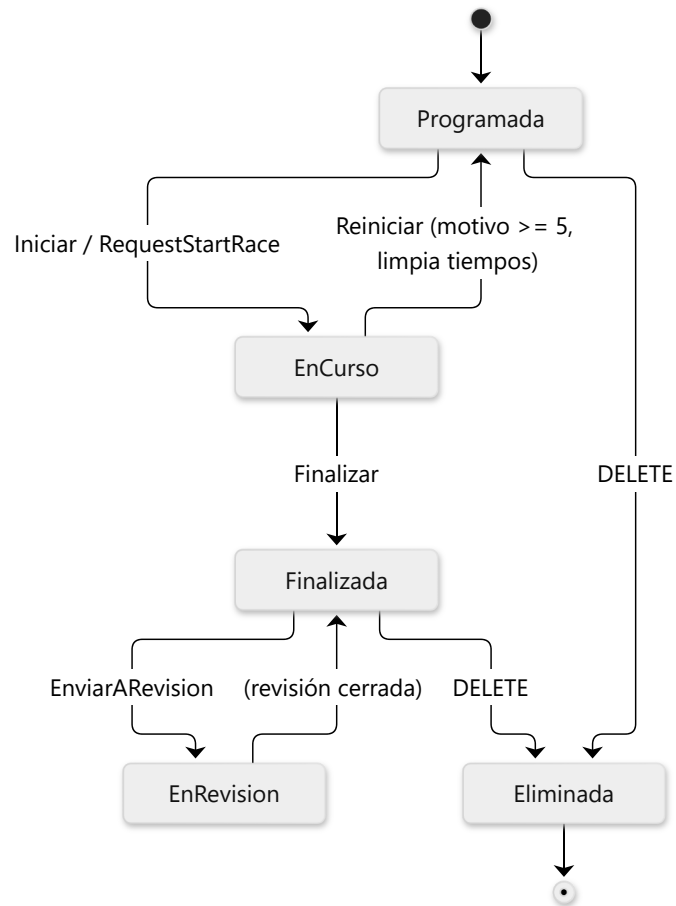
Resolución del token (OnMessageReceived): `Authorization: Bearer` → `?access_token` (SignalR) → cookie `X-Access-Token`.

2. Ciclo de vida evento / fase / resultado

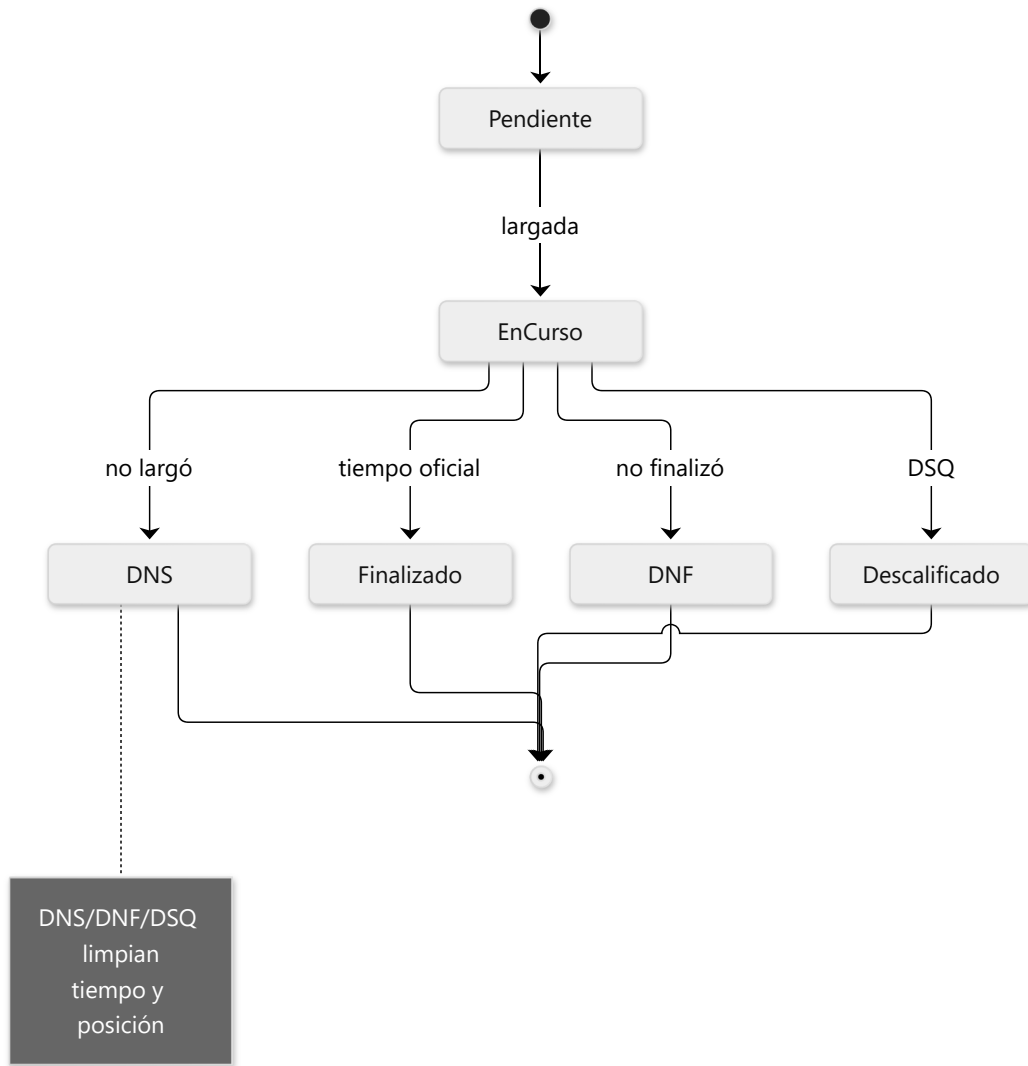
2.1 Estado del evento



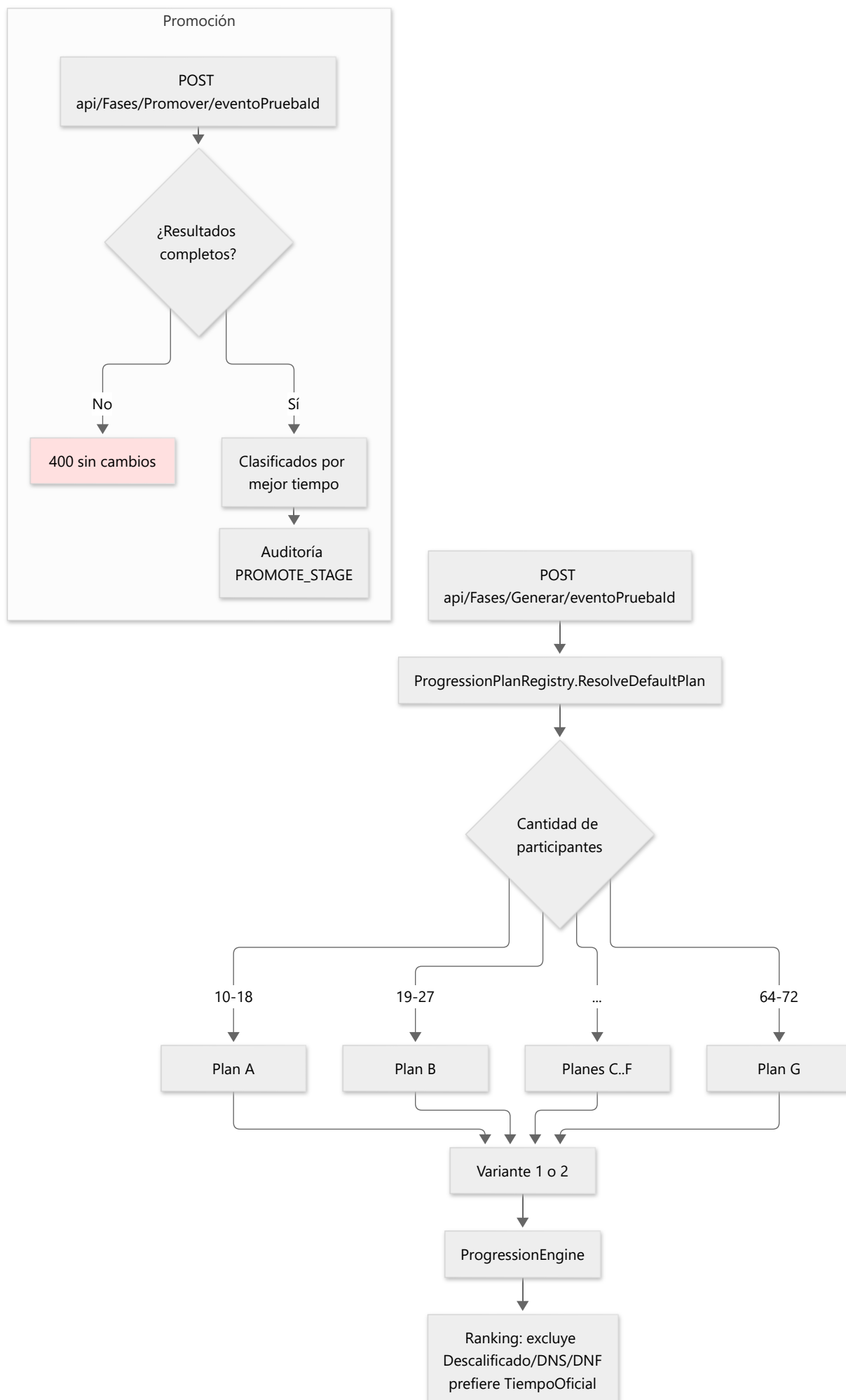
2.2 Estado de la fase

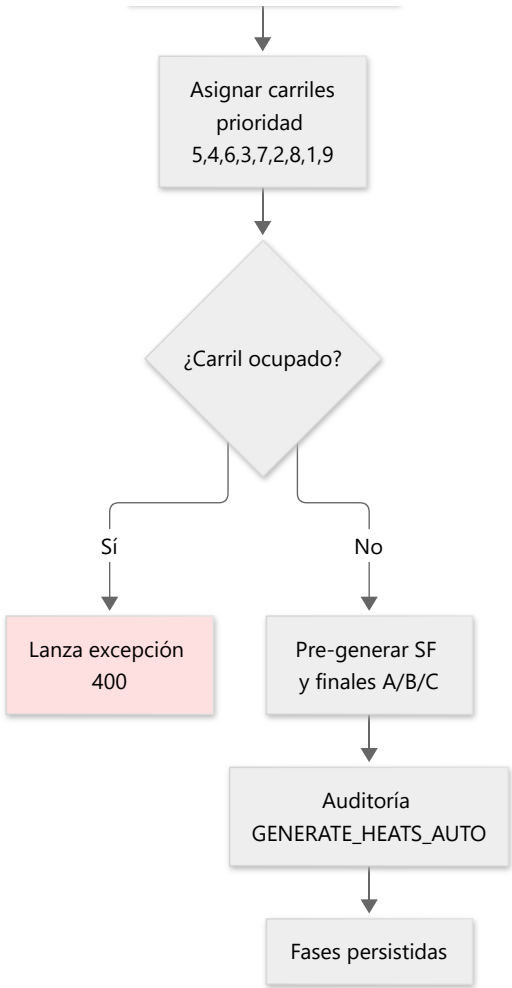


2.3 Estado del resultado



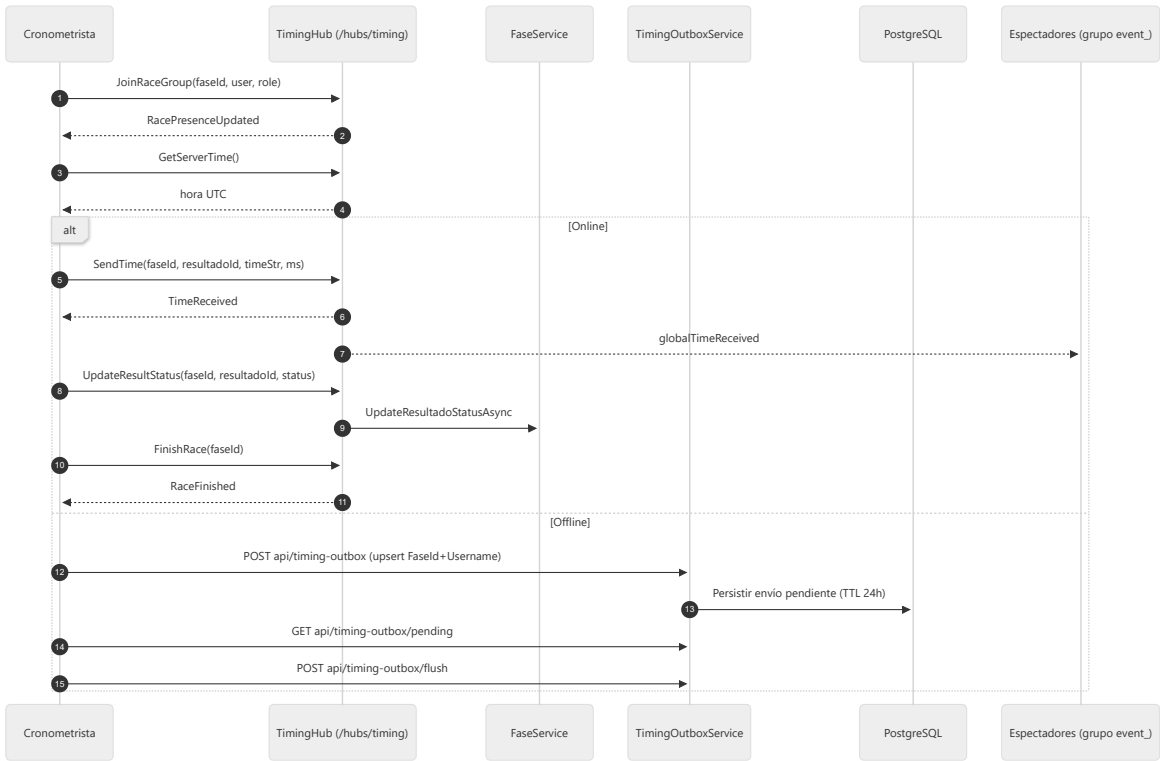
3. Motor de progresión ICF



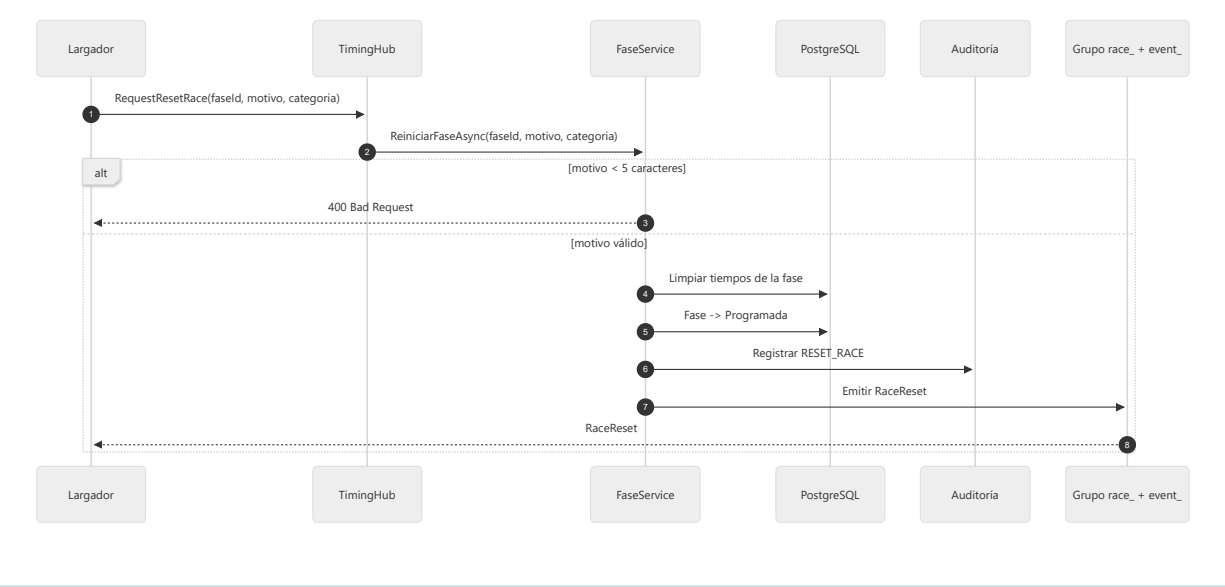


Alias: PlanX (comodín). **Maratón:** GenerarLargadaMaraton crea una única fase Largada con Carril = NumeroCompetidor y audita GENERATE_LARGADA_MARATON .

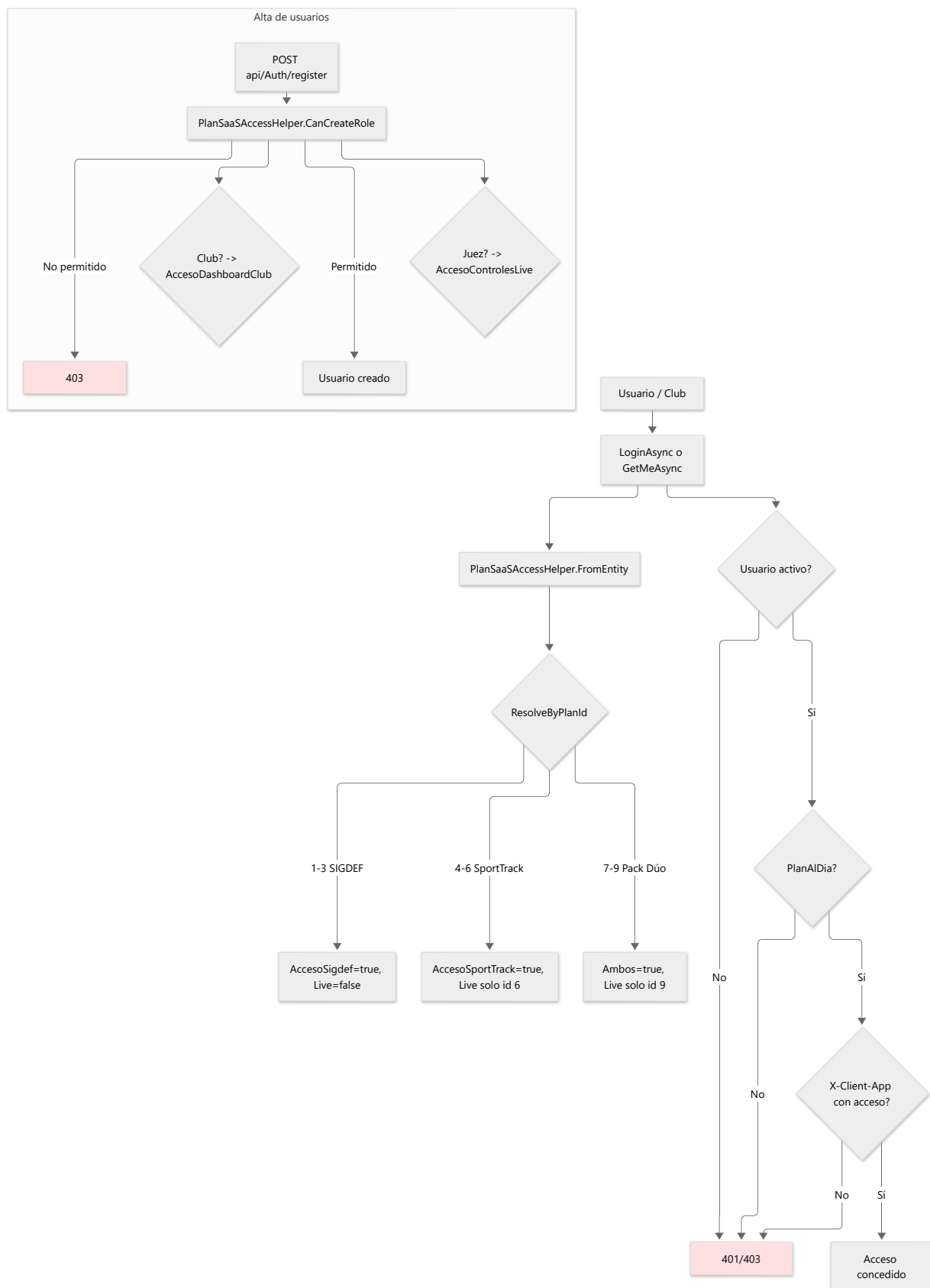
4. Cronometraje SignalR + outbox



5. Reseteo de regata

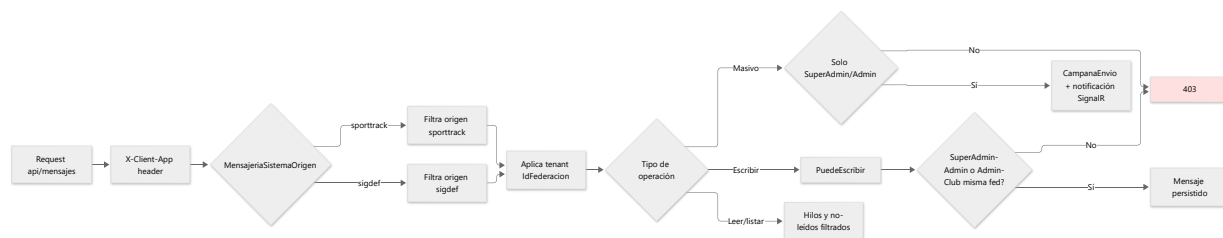


6. Enforcement de planes SaaS



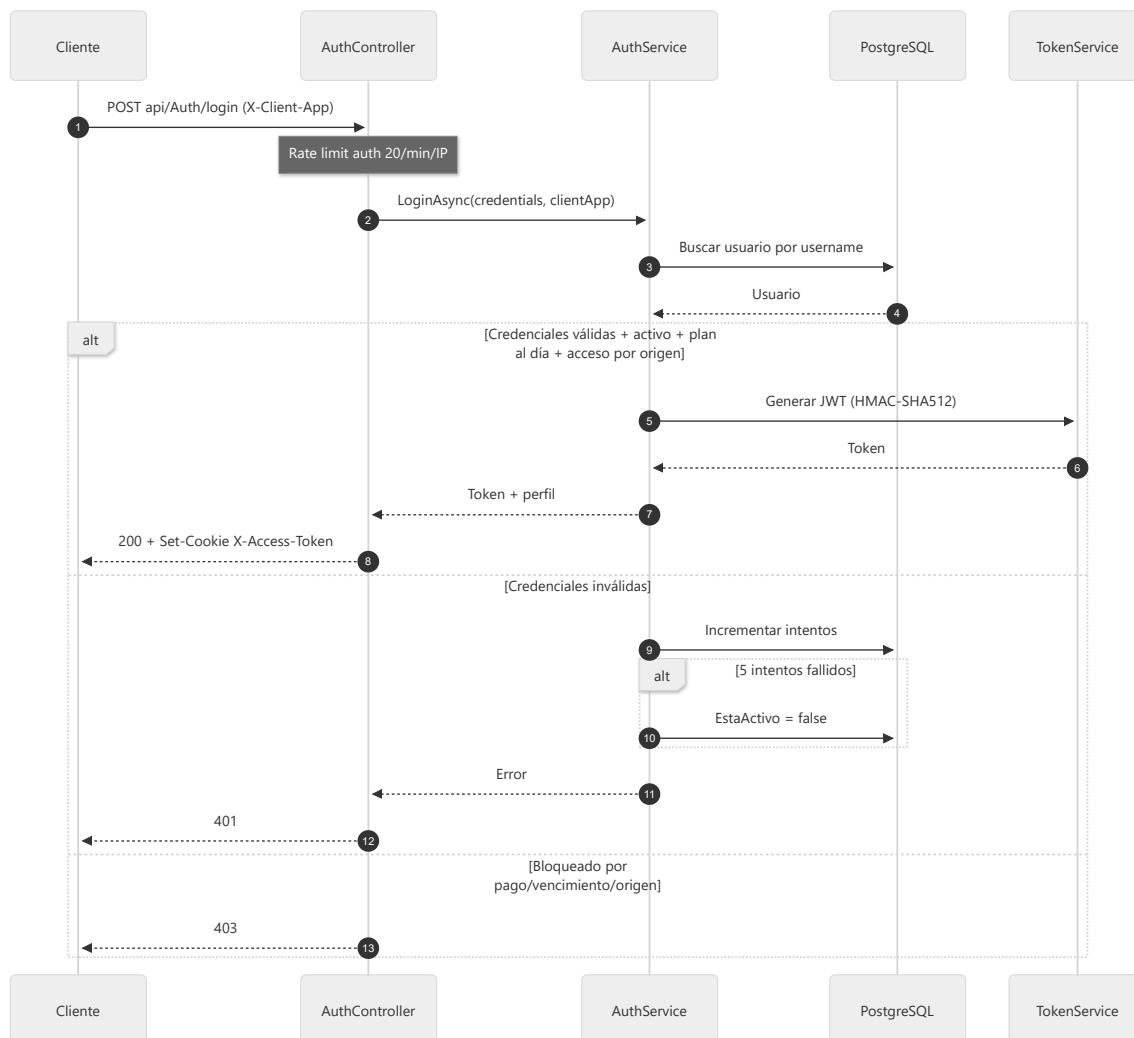
⚠ **Deuda:** MaxAtletas no se aplica aún en el alta de atletas; MaxTorneosActivos quedó en -1 (sin enforcement efectivo).

7. Mensajería aislada por sistema de origen

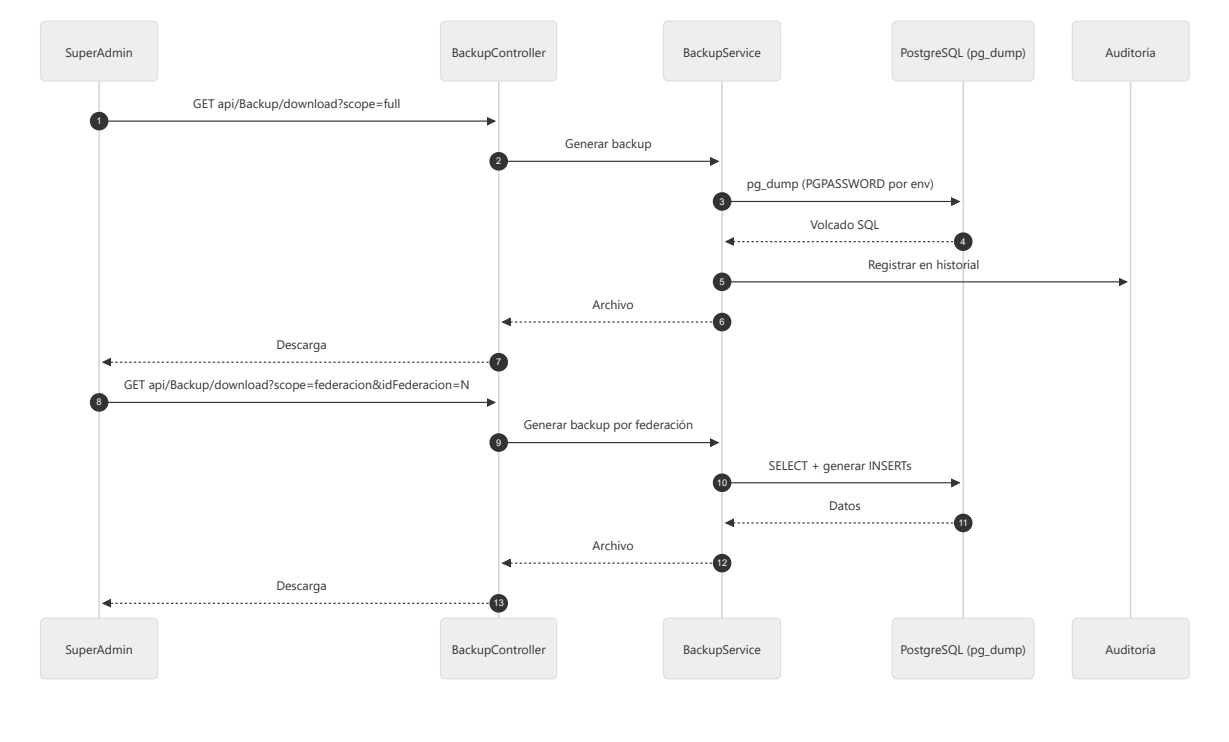


Notificación: `newMessageReceived` → grupo `user_{username}`.

8. Secuencia de login



9. Backup



Pie de página

- [04 · Manual de Uso de la API](#)
- [05 · Manual Técnico](#)
- [02 · Casos de Uso · Volver al índice del Pack](#)

04 · Manual de Uso de la API

Última actualización: 2026-09-27 Audiencia: integradores frontend, administradores y personal de soporte.

Índice

- 1. Convenciones
- 2. Autenticación
- 3. Consumo por caso de uso
- 4. Ejemplos (curl / PowerShell)
- 5. Swagger en desarrollo
- 6. Manejo de errores
- 7. Uso para soporte
- 8. FAQ

1. Convenciones

Aspecto	Valor
Base URL local	http://localhost:5012 / https://localhost:7232
Base URL producción	Dominio del servicio en Render (p. ej. https://<servicio>.onrender.com)
Prefijo	Rutas bajo api/** (el hub SignalR es /hubs/timing)
Formato	JSON (application/json); uploads multipart/form-data
Auth	Authorization: Bearer <JWT> (preferido) o cookie X-Access-Token
Cabecera de origen	X-Client-App: sporttrack sigdef (aislamiento de mensajería/logins)
Ciclos	Fechas/horas en UTC; el backend convierte DateTime a UTC globalmente
Rate limit	auth = 20 req/min/IP · live = 120 req/min/IP

2. Autenticación

2.1 Login y token

```
POST /api/Auth/login
Content-Type: application/json
X-Client-App: sigdef

{ "username": "admin", "password": "admin123" }
```

Respuesta 200 : body con el JWT y cabecera Set-Cookie: X-Access-Token=...

2.2 Resolución del token (orden real)

- 1. Authorization: Bearer <token>
- 2. Query string ?access_token=<token> (para SignalR)
- 3. Cookie X-Access-Token

2.3 Logout y reset

```
POST /api/Auth/logout
```

```
POST /api/Auth/solicitar-reset-password
{ "username": "..." }
```

La solicitud de reset devuelve siempre una **respuesta genérica** (no revela si el usuario existe).

2.4 Endpoints de identidad

Método	Ruta	Rol	Descripción
POST	api/Auth/login	Anónimo (rate auth)	Emite JWT + cookie
POST	api/Auth/logout	Anónimo	Limpia cookie
POST	api/Auth/solicitar-reset-password	Anónimo (rate auth)	Reset genérico
POST	api/Auth/register	Admins (rate auth)	Alta de usuario
GET	api/Auth/usuarios	Autenticado	Lista usuarios del ámbito
PUT	api/Auth/usuarios/{id}/password	Autenticado	Cambiar contraseña
PUT	api/Auth/usuarios/{id}/perfil	Autenticado	Actualizar perfil
PATCH	api/Auth/usuarios/{id}/toggle-activo	Admin, SuperAdmin, soporte	Activar/desactivar
DELETE	api/Auth/usuarios/{id}	Admin, SuperAdmin, soporte	Baja de login
GET	api/Auth/me	Autenticado	Perfil propio

3. Consumo por caso de uso

3.1 Live público (sin token)

Método	Ruta	Rol	Parámetros
GET	api/Eventos/proximos	Anónimo	—
GET	api/Eventos/{id}	Anónimo (rate live)	id
GET	api/Eventos/{id}/fases	Anónimo (rate live)	id
GET	api/Eventos/{id}/pruebas	Anónimo (rate live)	id
GET	api/Resultados/Fase/{faseId}	Anónimo (rate live , cache 30 s)	faseId
GET	api/Fases/EventoPrueba/{eventoPruebaId}	Anónimo (rate live)	eventoPruebaId
GET	api/Fases/all-by-evento/{eventoId}	Anónimo (rate live)	eventoId
GET	api/Health · api/Health/db	Anónimo	—
GET	api/SaaS/planes	Anónimo	—

3.2 Eventos y programación

Método	Ruta	Rol	Parámetros
GET	api/Eventos	Autenticado	filtros
GET	api/Eventos/debug	Autenticado	—
POST	api/Eventos	Autenticado	body evento
PUT	api/Eventos/{id}	Autenticado	id , body
DELETE	api/Eventos/{id}	Autenticado	id
POST	api/Eventos/{id}/pruebas	Autenticado	id , body prueba
POST	api/Eventos/{id}/pruebas/largada	Autenticado	id , body largada
PUT	api/Eventos/pruebas/{id}	Autenticado	id , body
DELETE	api/Eventos/pruebas/{id}	Autenticado	id

3.3 Fases y progresión ICF

Método	Ruta	Rol	Parámetros
POST	api/Fases/Generar/{eventoPruebaId}	CompetitionOperators	eventoPruebaId
POST	api/Fases/GenerarManual/{eventoPruebaId}	CompetitionOperators	eventoPruebaId , body
POST	api/Fases/GenerarLargadaMaraton	CompetitionOperators	body
POST	api/Fases/Promover/{eventoPruebaId}	CompetitionOperators	eventoPruebaId
POST	api/Fases/BatchUpdate	CompetitionOperators	body (lista)
GET	api/Fases/ProgresionAudit/{eventoPruebaId}	CompetitionOperators	eventoPruebaId
POST	api/Fases/{id}/Iniciar	CompetitionOperators	id , body (hora)

Método	Ruta	Rol	Parámetros
POST	api/Fases/{id}/Finalizar	CompetitionOperators	id
POST	api/Fases/{id}/Reiniciar	CompetitionOperators	id , motivo (≥5)
POST	api/Fases/{id}/EnviarARvision	CompetitionOperators	id
DELETE	api/Fases/{id}	CompetitionOperators	id

3.4 Resultados y cronometraje

Método	Ruta	Rol	Parámetros
GET	api/Resultados/Fase/{faseId}	Anónimo (rate live)	faseId
PUT	api/Resultados/BatchUpdate	CompetitionOperators	body (lista de resultados)
POST	api/timing-outbox	CompetitionOperators	body envío
GET	api/timing-outbox/pending	CompetitionOperators	—
POST	api/timing-outbox/flush	CompetitionOperators	—
POST	api/timing-outbox/{faseId}/commit	CompetitionOperators	faseId
DELETE	api/timing-outbox/{faseId}	CompetitionOperators	faseId

3.5 Inscripciones, participantes y catálogos

Método	Ruta	Rol
GET/POST/PUT/DELETE	api/Inscripciones , api/Inscripciones/{id} , api/Inscripciones/registro , api/Inscripciones/evento-prueba/{id} , api/Inscripciones/evento/{id}/club/{id}	Autenticado
PATCH	api/Inscripciones/{id}/toggle-seeding	Autenticado
CRUD	api/Participantes , api/Participantes/club/{clubId}	Autenticado
CRUD	api/Clubes , api/Botes , api/Categorias , api/Distancias	Autenticado
GET/POST/PUT/DELETE	api/legacy/eventos/{idEvento}/pruebas	Autenticado

3.6 SIGDEF

Método	Ruta	Rol	Notas
GET	api/Atleta , api/Atleta/paged , api/Atleta/{id} , api/Atleta/club/{clubId}	Autenticado	Listados y filtros
POST	api/Atleta , api/Atleta/full	Autenticado	Alta simple / alta completa
POST	api/Atleta/{id}/liberar	Autenticado	Pasa a Agente Libre
CRUD	api/Club , api/DelegadoClub , api/Entrenador , api/Tutor , api/Persona , api/Rol , api/Usuario , api/AtletaTutor , api/Inscripcion	Autenticado	CRUD estándar
GET	api/Traspaso/periodos , api/Traspaso/periodo-activo , api/Traspaso , api/Traspaso/{id} , api/Traspaso/{id}/validaciones , api/Traspaso/buscar-atletas , api/Traspaso/auditoria , api/Traspaso/export/csv	Autenticado	Consultas
POST	api/Traspaso , api/Traspaso/periodos , api/Traspaso/{id}/aceptar-origen , .../rechazar-origen , .../aprobar , .../rechazar , .../cancelar	Autenticado	Flujo de traspaso
PUT	api/Traspaso/periodos/{id}	Autenticado	Editar periodo
POST	api/Documentacion/upload (multipart, ≤6 MB)	Autenticado	Cloudinary
GET	api/Documentacion/persona/{id}	Autenticado	—
DELETE	api/Documentacion/{id}	Autenticado	—

3.7 Pagos y MercadoPago

Método	Ruta	Rol
GET	api/Pagos/historial	Autenticado
POST	api/Pagos/registrar	Autenticado
PUT	api/Pagos/clubes/{id}/toggle , api/Pagos/clubes/{id}/solicitar-pago	Autenticado
PUT	api/Pagos/atletas/{id}/toggle , api/Pagos/inscripciones/{id}/toggle	Autenticado
DELETE	api/Pagos/{id:int} , api/Pagos/bulk	Autenticado

Método	Ruta	Rol
POST	api/PagoTransaccion/preferencia + CRUD	Autenticado
POST	api/Notificacion/webhook	Anónimo ⚠ sin validación de firma

3.8 Mensajería

Método	Ruta	Rol
GET	api/mensajes/hilos , api/mensajes/hilos/{id} , api/mensajes/no-leidos/count , api/mensajes/campanas , api/mensajes/campanas/{id}	SuperAdmin, Admin, Club
POST	api/mensajes/hilos , api/mensajes/hilos/{id}/responder	SuperAdmin, Admin, Club
PATCH	api/mensajes/hilos/{id}/leer	SuperAdmin, Admin, Club
POST	api/mensajes/hilos/masivo	SuperAdmin, Admin

3.9 SaaS

Método	Ruta	Rol
GET	api/SaaS/planes	Anónimo
GET	api/SaaS/debug-me	SuperAdmin, soporte
PUT	api/SaaS/planes/{id}	SuperAdmin
POST	api/SaaS/asignar-plan	SuperAdmin, Admin
GET	api/SaaS/clubes-status	SuperAdmin, Admin, soporte
PATCH	api/SaaS/clubes/{id}/toggle-activo	SuperAdmin, Admin, soporte
POST	api/SaaS/create-federacion	SuperAdmin
GET	api/SaaS/global-metrics	SuperAdmin, soporte

3.10 Auditoría, soporte, backups, audience

Método	Ruta	Rol
GET	api/Auditoria , api/Auditoria/por-eventos	Autenticado
POST	api/Auditoria/client-action	Autenticado
DELETE	api/Auditoria/por-evento/{eventoId}/sin-problemas , api/Auditoria/{id}	Autenticado
GET	api/Diagnostic/check-eventos , api/Diagnostic/search/{query}	Admin, SuperAdmin, soporte
GET	api/Support/por-eventos , api/Support/timing-outbox , api/Support/logs	Autenticado
POST	api/Support/client-action , api/Support/timing-outbox/{faseId}/commit , api/Support/frontend-error	Autenticado / Anónimo
DELETE	api/Support/timing-outbox/{id} , api/Support/logs/clear	Autenticado / SuperAdmin
GET	api/Backup/download?scope=full federacion&idFederacion=N , api/Backup/history	SuperAdmin, soporte
GET	api/Audience/live , api/Audience/peaks , api/Audience/capacity	SuperAdmin, soporte
PUT	api/Audience/capacity	SuperAdmin, soporte

3.11 SignalR (tiempo real)

Elemento	Valor
Hub	/hubs/timing
Grupos	race_{faseId} , event_{eventoId} , operators , user_{username} , fed_{federacionId}
Métodos AllowAnonymous	JoinRaceGroup , JoinEventGroup , LeaveRaceGroup , GetServerTime
Métodos CompetitionOperators	JoinOperatorsGroup , RequestStartRace , RequestResetRace , RecordLap , FinishRace , SendTime , UpdateResultStatus
Otros	JoinUserNotificationsGroup , JoinFederationNotificationsGroup , RequestPaymentStatusChange

Elemento	Valor
Eventos	RaceStarted, RaceFinished, RaceReset, RaceInReview, globalRaceStarted, globalRaceOfficialized, globalRaceInReview, GlobalResultStatusUpdated, TimeReceived, globalTimeReceived, LapRecorded, ResultadoActualizado, RacePresenceUpdated, EventPresenceUpdated, newMessageReceived, newEventCreated, paymentStatusChangeRequested

Conexión con token: `?access_token=<JWT>`.

4. Ejemplos (curl / PowerShell)

4.1 Login (curl)

```
curl -X POST http://localhost:5012/api/Auth/login \
-H "Content-Type: application/json" \
-H "X-Client-App: sigdef" \
-d '{"username":"admin","password":"admin123"}' -i
```

4.2 Login (PowerShell) y guardado del token

```
$body = @{ username = "admin"; password = "admin123" } | ConvertTo-Json
$resp = Invoke-RestMethod -Method Post -Uri "http://localhost:5012/api/Auth/login" `
-ContentType "application/json" -Headers @{ "X-Client-App" = "sigdef" } -Body $body
$token = $resp.token
$headers = @{ Authorization = "Bearer $token"; "X-Client-App" = "sigdef" }
```

El nombre exacto del campo del token depende del DTO de login (AuthService); inspeccioná el body de respuesta con Swagger o en el frontend.

4.3 Consultar Live (anónimo)

```
curl http://localhost:5012/api/Resultados/Fase/123
```

4.4 Generar fases

```
Invoke-RestMethod -Method Post -Uri "http://localhost:5012/api/Fases/Generar/45" -Headers $headers
```

4.5 Enviar resultados en lote

```
curl -X PUT http://localhost:5012/api/Resultados/BatchUpdate \
-H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
-d '["{"resultadoId":1,"tiempo":"00:01:32.450","posicion":1,"estado":"Finalizado"}"]'
```

4.6 Encolar/Enviar timing outbox

```
curl -X POST http://localhost:5012/api/timing-outbox \
-H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
-d '{"faseId":123,"resultadoId":9,"tiempo":"00:02:10.100"}'
```

4.7 Descargar backup (PowerShell)

```
Invoke-WebRequest -Uri "https://<servicio>.onrender.com/api/Backup/download?scope=full" `
-Headers $headers -OutFile "backup.sql"
```

5. Swagger en desarrollo

- Disponible **solo** con `ASPNETCORE_ENVIRONMENT=Development`.
- URL: `http://localhost:5012/swagger`
- Botón **Authorize** con esquema `Bearer`: pegar el JWT (sin la palabra `Bearer` según la configuración del esquema).
- En producción Swagger **no** se expone.

6. Manejo de errores

Código	Significado	Causas típicas	Acción
400	Bad Request	Validación, carril ocupado, motivo de reseteo < 5, resultados incompletos	Revisar body/parámetros
401	Unauthorized	Sin token, token expirado, usuario inactivo/bloqueado, login inválido	Renovar login
403	Forbidden	Rol sin permiso, plan sin acceso, aislamiento de tenant	Verificar rol y plan
404	Not Found	Recurso inexistente	Verificar id
429	Too Many Requests	Rate limit <code>auth</code> (20/min) o <code>live</code> (120/min)	Reintentar con backoff
500	Internal Server Error	Excepción no controlada (se audita <code>ERROR_FATAL</code> , mensaje sanitizado)	Ver logs <code>/api/Support/logs</code>

`ExceptionHandler` traduce `NotFound` / `Unauthorized` / `BadRequest`; las inesperadas se loguean, se auditan como `ERROR_FATAL` y devuelven mensaje sanitizado.

7. Uso para soporte

7.1 Timing outbox

Acción	Endpoint
Ver pendientes de una fase/contexto	<code>GET api/Support/timing-outbox</code> o <code>GET api/timing-outbox/pending</code>
Commit manual de una fase	<code>POST api/Support/timing-outbox/{faseId}/commit</code>
Descartar una entrada	<code>DELETE api/Support/timing-outbox/{id}</code>

7.2 Auditoría

Acción	Endpoint
Consultar registros	<code>GET api/Auditoria</code>
Ver agrupado por eventos (últimos 2500 registros)	<code>GET api/Auditoria/por-eventos</code>
Registrar acción de cliente	<code>POST api/Auditoria/client-action</code>
Limpiar sin problemas de un evento	<code>DELETE api/Auditoria/por-evento/{eventoId}/sin-problemas</code>

7.3 Logs

Acción	Endpoint
Ver logs	<code>GET api/Support/logs</code>
Limpiar logs	<code>DELETE api/Support/logs/clear</code> (SuperAdmin)
Reportar error de frontend	<code>POST api/Support/frontend-error</code> (Anónimo)

7.4 Métricas y audiencia

Acción	Endpoint
Métricas globales	<code>GET api/SaaS/global-metrics</code>
Audiencia live / picos / capacidad	<code>GET api/Audience/live, /peaks, /capacity</code>
Ajustar capacidad	<code>PUT api/Audience/capacity</code>
Diagnóstico de eventos	<code>GET api/Diagnostic/check-eventos</code>
Búsqueda de diagnóstico	<code>GET api/Diagnostic/search/{query}</code>

7.5 Backups

Acción	Endpoint
Backup completo	<code>GET api/Backup/download?scope=full</code>
Backup por federación	<code>GET api/Backup/download?scope=federacion&idFederacion=N</code>
Historial	<code>GET api/Backup/history</code>

8. FAQ

¿Por qué recibo 401? Token ausente/expirado, usuario inactivo o bloqueado por pago/vencimiento, o credenciales inválidas. Tras 5 intentos fallidos el usuario queda `EstaActivo = false`. Verificá también la cabecera `X-Client-App`: si el sistema de origen no tiene acceso al plan, el login puede fallar.

¿Por qué recibo 429? Superaste el rate limit. Los endpoints `auth` admiten 20 req/min/IP y los `live` 120 req/min/IP. Esperá ~1 minuto o reducí la frecuencia (usá el Live solo cuando haga falta).

¿Cómo funciona el reset de contraseña? `POST api/Auth/solicitar-reset-password` devuelve siempre una respuesta genérica. El backend notifica automáticamente (mensajería) si el usuario existe.

¿Cómo descargo un backup? Con rol SuperAdmin/soporte llamá a `GET api/Backup/download?scope=full`. Requiere `pg_dump` disponible en el entorno (en Docker viene `postgresql-client-18`).

¿Cómo se comporta el Live sin login? Los endpoints marcados `[AllowAnonymous]` (resultados, fases, próximos) son públicos con rate limit `live` y cache corto. El hub SignalR se conecta anónimo para lectura, pero las escrituras exigen rol `CompetitionOperators`.

¿Cuál es la diferencia entre cookie y Bearer? Ambos funcionan. Se recomienda `Authorization: Bearer` por ser cross-origin seguro; la cookie `X-Access-Token` se mantiene como compatibilidad y para escenarios de navegador.

Pie de página

- [📄 05 · Manual Técnico](#)
- [📄 06 · Manual de Instalación y Despliegue](#)
- [📄 03 · Diagramas de Flujo · Volver al índice del Pack](#)

05 · Manual Técnico

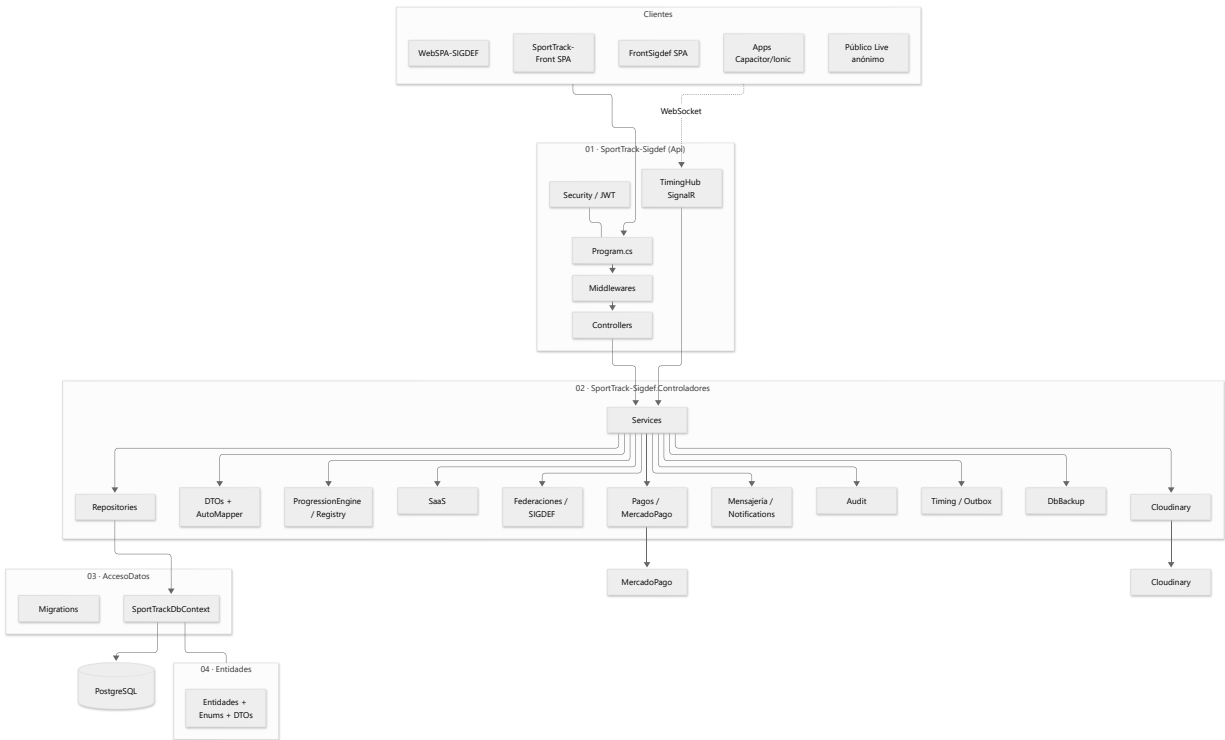
Última actualización: 2026-09-27 Audiencia: desarrolladores backend, DevOps y mantenimiento.

Índice

- 1. Arquitectura
- 2. Stack real
- 3. Estructura de la solución
- 4. Arranque y Program.cs
- 5. Catálogo de controllers (rutas y roles)
- 6. Servicios clave
- 7. Modelo de datos
- 8. Seguridad
- 9. SignalR
- 10. Observabilidad y auditoría
- 11. Backups
- 12. Docker y despliegue
- 13. Deuda técnica y discrepancias conocidas
- 14. Migraciones y seed
- 15. Estrategia offline / outbox
- 16. Testing y mantenimiento

1. Arquitectura

Arquitectura en capas con separación por proyectos y comunicación por interfaces (Services/Repositories + DI).



2. Stack real

Componente	Versión / detalle
Framework	.NET 10 (net10.0) — recién migrado desde .NET 8
Lenguaje	C#
Web	ASP.NET Core Web API

Componente	Versión / detalle
ORM	EF Core + Npgsql 10.0.3
Base de datos	PostgreSQL 14+ (Render PG 18.x)
Tiempo real	SignalR
Auth	JWT HMAC-SHA512; BCrypt para contraseñas
Mapeo	AutoMapper (MappingProfile)
Cache	IMemoryCache + ILiveCacheService (TTL 30–45 s)
Archivos	Cloudinary (CloudinarySettings) con fallback local
Pagos	mercadopago-sdk
Serialización	System.Text.Json con ReferenceHandler.IgnoreCycles
Codificación	UTF-8 forzado por Directory.Build.props

3. Estructura de la solución

SportTrack-Sigdef.sln — 4 proyectos:

Proyecto	Rol	Contenido clave
01 · SportTrack-Sigdef	Api / composición	Program.cs , Controllers/** , Middleware/** , Security/** , CORS, JWT, Swagger, SignalR
02 · SportTrack-Sigdef.Controladores	Lógica de aplicación	Services/Repositories , DTOs , Mappings (AutoMapper), Hubs , SaaS , motor de progresión ICF (Fase/Progression), Federaciones (SIGDEF), Pago , PagosSIGDEF , Mensajes , Notifications , Audience , Audit , Timing , DbBackup , Cloudinary, mercadopago-sdk
03 · SportTrack-Sigdef.AccesoDatos	Persistencia	SportTrackDbContext + Migrations , Npgsql
04 · SportTrack-Sigdef.Entidades	Dominio	Entidades, enums, DTOs

Estructura de carpetas relevante:

SportTrack-Sigdef/	
└─ SportTrack-Sigdef/	(Api)
└─ Program.cs	
└─ appsettings.json	
└─ Dockerfile → (raíz)	
└─ Controllers/	(Auth, Eventos, Fases, Resultados, ...)
└─ SIGDEF/	(Atleta, Club, Tutor, Traspaso, ...)
└─ Middleware/	(Exception, SecurityHeaders)
└─ SportTrack-Sigdef.Controladores/	
└─ Fase/Progression/	(ProgressionEngine, PlanRegistry)
└─ SaaS/	(SaaSService, PlanSaaSAccessHelper)
└─ Federaciones/	(AltaAtletaService, TraspasoService, ...)
└─ Hubs/	(TimingHub, TimingGroups)
└─ Timing/, Audit/, DbBackup/, Audience/, Mensajes/, Notifications/	
└─ SportTrack-Sigdef.AccesoDatos/	
└─ SportTrack-Sigdef.Entidades/	

4. Arranque y Program.cs

4.1 Cadena de conexión

ResolveConnectionString : usa ConnectionStrings.DefaultConnection ; si no, la variable DATABASE_URL , normalizando postgres:// → postgresql:// .

4.2 Servicios registrados (resumen)

- AddDbContext<SportTrackDbContext> con UseNpgsql → EnableRetryOnFailure(3) + CommandTimeout(30) .
- AddMemoryCache (SizeLimit = 1024) + ILiveCacheService .
- Audience: IAudiencePresenceTracker, IAudienceCapacitySettings, IAudienceMetricsService, AudienceSnapshotBackgroundService (hosted).

- `AddSignalR`.
- CORS whitelist (`AllowedOrigins` + localhost dev + dominios propios).
- JWT con `TokenKeyResolver`; `OnMessageReceived` (Bearer → query → cookie).
- `FallbackPolicy` = `RequireAuthenticatedUser`.
- `RateLimiter` `auth` (20/min) y `live` (120/min).
- `AddControllers().AddApplicationPart(Controladores)` + `ReferenceHandler.IgnoreCycles`.
- Swagger solo en Development con `Bearer`.
- `UseForwardedHeaders` (limpia `KnownIPNetworks` / `KnownProxies` para Render).
- Migraciones automáticas al arranque + `EventoEstadoSyncService.SyncAllAsync`.
- `EventoEstadoBackgroundService` (hosted, 15 min).

4.3 Pipeline

```
ExceptionMiddleware
→ SecurityHeadersMiddleware
→ HSTS/HTTPS (no Dev)
→ Swagger (Dev)
→ CORS
→ RateLimiter
→ Authentication
→ Authorization
→ MapControllers
→ MapHub<TimingHub>("/hubs/timing").AllowAnonymous()
```

4.4 CORS (whitelist)

Orígenes por defecto: `localhost:3000/5173/5174`, `https://localhost`, `http://localhost`, `capacitor://localhost`, `ionic://localhost`, `sporttrack.pro`, `www.sporttrack.pro`, `sigdef.pro`, `www.sigdef.pro`, `sigdef.vercel.app`, `www.sigdef.vercel.app`, `dgotech.org`, `www.dgotech.org`, `sporttrack-fec.vercel.app`, `oficialsporttrack.vercel.app`. Con `AllowCredentials`. El `ExceptionMiddleware` **re-aplica** las cabeceras CORS para que las respuestas de error no las pierdan.

4.5 Middlewares

Middleware	Responsabilidad
<code>ExceptionMiddleware</code>	Traduce <code>NotFound</code> / <code>Unauthorized</code> / <code>BadRequest</code> ; inesperadas → log + auditoría <code>ERROR_FATAL</code> + mensaje sanitizado; re-aplica CORS
<code>SecurityHeadersMiddleware</code>	<code>X-Content-Type-Options</code> , <code>X-Frame-Options: DENY</code> , <code>Referrer-Policy</code> , <code>Permissions-Policy</code>

5. Catálogo de controllers (rutas y roles)

Ruta base `api/<Nombre>` salvo indicación. **Sin rol indicado = solo autenticado.**

5.1 Controllers de la Api

Controller	Ruta	Métodos principales (roles)
<code>AuthController</code>	<code>api/Auth</code>	login [Anónimo, rate auth], logout [Anónimo], solicitar-reset-password [Anónimo, rate auth], register [Admins, rate auth], usuarios [Auth], usuarios/{id}/password [Auth], usuarios/{id}/perfil [Auth], toggle-activo [Admin/SuperAdmin/soporte], DELETE usuarios/{id} [Admin/SuperAdmin/soporte], me [Auth]
<code>EventosController</code>	<code>api/Eventos</code>	GET [Auth], debug [Auth], {id}/fases [Anónimo+live], proximos [Anónimo], {id} [Anónimo+live], POST/PUT/DELETE [Auth], {id}/pruebas [Anónimo+live], POST {id}/pruebas [Auth], {id}/pruebas/largada [Auth], PUT/DELETE pruebas/{id} [Auth]
<code>FasesController</code>	<code>api/Fases</code>	EventoPrueba/{id} y all-by-evento/{id} [Anónimo+live]; ProgresionAudit, BatchUpdate, Generar, GenerarManual, GenerarLargadaMaraton, Promover, Iniciar, Finalizar, Reiniciar, EnviarARvision, DELETE [CompetitionOperators]
<code>ResultadosController</code>	<code>api/Resultados</code>	GET Fase/{id} [Anónimo+live, cache 30 s], PUT BatchUpdate [CompetitionOperators]
<code>InscripcionesController</code>	<code>api/Inscripciones</code>	GET registro, GET, GET {id}, GET evento-prueba/{id}, GET evento/{id}/club/{id}, POST, PUT {id}, DELETE {id}, PATCH {id}/toggle-seeding [Auth]
<code>ParticipantesController</code>	<code>api/Participantes</code>	CRUD + club/{clubId} [Auth]
<code>ClubesController</code>	<code>api/Clubes</code>	CRUD [Auth]

Controller	Ruta	Métodos principales (roles)
BotesController / CategoriaController / DistanciaController	api/Botes / api/Categorias / api/Distancias	CRUD [Auth]
PagosController	api/Pagos	historial, registrar, clubes/{id}/toggle, clubes/{id}/solicitar-pago, atletas/{id}/toggle, inscripciones/{id}/toggle, DELETE {id:int}, DELETE bulk [Auth]
SaaSController	api/SaaS	debug-me [SuperAdmin/soporte], planes [Anónimo], PUT planes/{id} [SuperAdmin], asignar-plan [SuperAdmin/Admin], clubes-status [SuperAdmin/Admin/soporte], PATCH clubes/{id}/toggle-activo [SuperAdmin/Admin/soporte], create-federacion [SuperAdmin], global-metrics [SuperAdmin/soporte]
AuditoriaController	api/Auditoria	GET, por-eventos, POST client-action, DELETE por-evento/{eventold}/sin-problemas, DELETE {id} [Auth]
DiagnosticController	api/Diagnostic	check-eventos, search/{query} [Admin/SuperAdmin/soporte]
AudienceController	api/Audience	live, peaks, capacity, PUT capacity [SuperAdmin/soporte]
BackupController	api/Backup	download?scope=full federacion&idFederacion=N, history [SuperAdmin/soporte]
TimingOutboxController	api/timing-outbox	POST, pending, flush, {faseId}/commit, DELETE {faseId} [CompetitionOperators]
HealthController	api/Health	GET, db [Anónimo]
MensajesController	api/mensajes	hilos, hilos/{id}, POST hilos, responder, PATCH leer, no-leidos/count [SuperAdmin/Admin/Club]; masivo, campanas, campanas/{id} [SuperAdmin/Admin]
SupportController	api/Support	por-eventos, client-action, timing-outbox, timing-outbox/{faseId}/commit, DELETE timing-outbox/{id}, logs [Auth]; frontend-error [Anónimo]; logs/clear [SuperAdmin]
FederacionesController	api/Federaciones	GET/{id} [Auth]; POST [SuperAdmin]; PUT [SuperAdmin/Admin]; DELETE [SuperAdmin]
WeatherForecastController	WeatherForecast	Legacy/template — pendiente de eliminar (deuda)

5.2 Controllers en el ensamblado Controladores (vía AddApplicationPart)

Controller	Ruta	Notas
PagoTransaccionController	api/PagoTransaccion	POST preferencia (MercadoPago) + CRUD
NotificacionController	api/Notificacion	POST webhook [Anónimo] ⚠ sin validación de firma
EventoPruebaController	api/legacy/eventos/{idEvento}/pruebas	Legacy

5.3 Controllers SIGDEF

Controller	Ruta	Métodos
AtletaController	api/Atleta	GET, GET {id}, GET paged, GET club/{clubId}, POST, POST full, PUT {id}, POST {id}/liberar, DELETE {id}
AtletaTutorController	api/AtletaTutor	CRUD + atleta/{id}, tutor/{id}
ClubController	api/Club	CRUD + search/{term}
DelegadoClubController	api/DelegadoClub	CRUD + federacion/{idFederacion}
DocumentacionController	api/Documentacion	POST upload, GET persona/{id}, DELETE {id}
EntrenadorController	api/Entrenador	CRUD + seleccion
InscripcionController	api/Inscripcion	GET, {id}, evento/{idEvento}, POST, DELETE
PersonaController	api/Persona	CRUD + documento/{documento}
RolController	api/Rol	CRUD
TraspasoController	api/Traspaso	periodos, periodo-activo, buscar-atletas, {id}, {id}/validaciones, POST, aceptar-origen, rechazar-origen, aprobar, rechazar, cancelar, auditoria, export/csv
TutorController	api/Tutor	CRUD
UsuarioController	api/Usuario	CRUD + {id}/change-password

6. Servicios clave

6.1 Motor de progresión ICF

Componente	Responsabilidad
ProgressionPlanRegistry	Planes A1-G2, SlotRule / BtSlotRule , alias PlanX , validación de carriles duplicados, ResolveDefaultPlan por cantidad (10-18→A, 19-27→B, ..., 64-72→G; variante 1/2)
ProgressionEngine	Rankings (excluye Descalificado / DNS / DNF , prefiere TiempoOficial); eliminatorias/semifinales → finales A/B/C por mejores tiempos; Assign lanza si el carril está ocupado
FaseService	GenerarFasesAutoAsync (9 carriles, prioridad [5,4,6,3,7,2,8,1,9] , cabezas de serie, pre-genera SF/finales, audita GENERATE_HEATS_AUTO); GenerarFasesManualAsync ; GenerarLargadaMaratonAsync (fase Largada , Carril = NumeroCompetidor , audita GENERATE_LARGADA_MARATON); PromoverFasesAsync (valida resultados completos, audita PROMOTE_STAGE); Iniciar / Finalizar (Posición+Oficial)/ Reiniciar (motivo ≥5, limpia tiempos, vuelve a Programada , audita RESET_RACE)/ EnviarAREvision ; BatchUpdateFasesAsync ; GetProgressionAuditAsync . Zona horaria del evento con fallback Argentina UTC-3

6.2 SaaS

Componente	Responsabilidad
SaaSService	GetPlanesAsync , UpdatePlanAsync (PrecioAnual = Precio×12×(1-d/100) , MaxTorneosActivos=-1), AsignarPlanAClubAsync , GetClubesStatusAsync (atletas usados vs MaxAtletas , PlanAlDia), ToggleClubActivoAsync , CreateFederacionWithAdminAsync (mapea 23505), GetGlobalMetricsAsync
PlanSaaSAccessHelper	IDs 1-3 SIGDEF, 4-6 SportTrack, 7-9 Pack Dúo; AccesoControlesLive solo 6 y 9; CanCreateRole (Club ← AccesoDashboardClub ; jueces ← AccesoControlesLive); IsJudgeRole = Largador/Cronometrista/JuezControl/ControlTecnico

6.3 SIGDEF / federación

Componente	Responsabilidad
AltaAtletaService	NormalizarDocumento , BuscarPorDocumentoAsync (3 estrategias), UpsertParticipanteAsync , EnsureAtletaFederacionAsync , AltaAtletaCompletaAsync , ResolverFederacionIdAsync
TraspasoService + TraspasoNotificacionService	Flujo de traspasos + notificaciones
TenantScopeHelper / TenantProvider	Aislamiento multi-tenant por federación

6.4 Mensajería y notificaciones

Componente	Responsabilidad
MensajeService + MensajeriaSistemaOrigen	Aislamiento por X-Client-App ; filtra hilos/campañas/no-leídos; PuedeEscribir (SuperAdmin↔Admin, Admin↔Club misma federación); masivo solo SuperAdmin/Admin; notificación automática + reset password
NotificationBroadcastService	newMessageReceived → user_{username} ; newEventCreated → fed_{federacionId}

6.5 Auditoría

AuditService , AuditEventCardsQuery (últimos 2500 registros agrupados por evento), AuditLegacyScopeResolver .

6.6 Timing / sesión de cronometrista

SessionLifetimePolicy (24 h para Cronometrista vs 5 h otros), TimingOutboxService (TTL 24 h; upsert por (FaseId, Username) ; CommitAsync aplica BatchUpdate + FinalizarFase / EnviarAREvision ; FlushPendingAsync ; PurgeExpiredAsync).

6.7 Otros

Componente	Responsabilidad
EventoEstadoSyncService	Programada→EnCurso→Finalizado por día local, nunca retrocede; background 15 min
BackupService	full con pg_dump (PGPASSWORD por env); federacion con INSERTs; historial desde Auditoría
ResultadoBatchUpdateService	Limpia tiempo/posición en DNS/DNF/DSQ; audita SAVE_TIMING ; emite ResultadoActualizado
LiveCacheService	Cache TTL 30-45 s
DocumentacionService	Upload a Cloudinary con fallback local; reglas FileUploadRules (6 MB; jpg/jpeg/png/webp/gif/pdf)

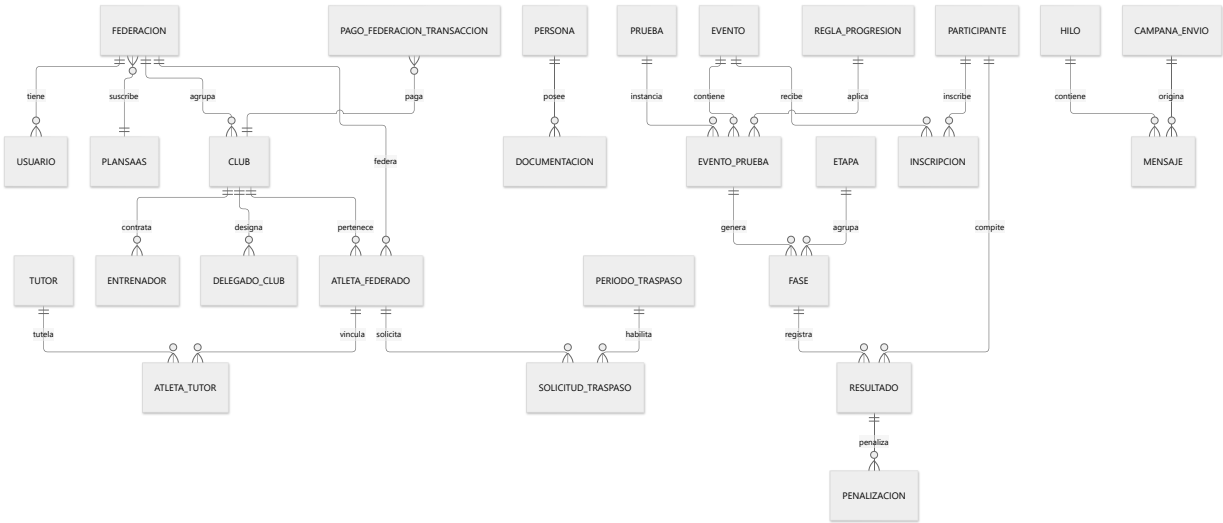
7. Modelo de datos

SportTrackDbContext — DateTime → UTC global, enums persistidos como string.

7.1 Esquemas

Esquema	Entidades
seguridad	Usuarios
catalogos	Sexos, Botes, Categorias, Distancias, Clubes, PlanesSaaS
federacion	Federaciones, DelegadosClub, Entrenadores, Tutores, AtletasFederados, AtletasTutores, Roles, DocumentacionPersonas, PagosTransacciones, PeriodosTraspaso, SolicitudesTraspaso
regatas	Eventos, Pruebas, EventoPruebas, Participantes, Inscripciones, InscripcionTripulantes, Etapas, Fases, ReglasProgresion, Resultados, Penalizaciones, Pagos
comunicacion	Hilos, Mensajes, CampanasEnvio
public	Auditoria, AudiencePeakSnapshots, AudienceMonitorSettings, TimingSubmissionOutbox

7.2 Diagrama ER (núcleo)



Nombres de entidad en el diagrama en formato legible; en código usan PascalCase (`Federacion` , `EventoPrueba` , `AtletaFederacion` , `SolicitudTraspaso` , `TimingSubmissionOutbox` , ...).

7.3 Detalles de mapeo relevantes

- **Índices únicos:** `Usuario.Username` y `Email` ; `Participante.Email` y `Documento` (parcial); (`FaseId`, `Carril`); (`IdEvento`, `IdPrueba`, `FechaHora`) .
- `Resultado.TiempoOficial` es `interval` .
- `Auditoria.IdEvento` / `IdEventoPrueba` son `[NotMapped]` ; el scope se embebe en `Detalle` .
- Seed inicial: 3 `Sexos` , 6 `Botes` , 11 `Categorias` , 17 `Distancias` (incluye 6000 m id 17), 9 `PlanesSaaS` (SIGDEF/SportTrack/Pack Dúo S/M/L), usuario `admin/admin123` (SuperAdmin, BCrypt).

7.4 Conteo de entidades

El proyecto `Entidades/Entidades` contiene **38 archivos de entidad** y `Entidades/Enums` **22 archivos de enum** (algunos enums tienen alias duplicados por compatibilidad, p. ej. `CategoriaEdad` / `CategoriaEdadEnum`).

8. Seguridad

8.1 Roles

Constante	Valor
<code>RoIFederacion</code>	<code>SuperAdmin</code> , <code>Admin</code> , <code>Club</code> , <code>Largador</code> , <code>Cronometrista</code> , <code>JuezControl</code> , <code>ControlTecnico</code> , <code>soporte_tecnico</code>
<code>CompetitionOperators</code>	<code>Admin</code> , <code>SuperAdmin</code> , <code>JuezControl</code> , <code>Largador</code> , <code>Cronometrista</code> , <code>ControlTecnico</code> , <code>soporte_tecnico</code>
<code>Admins</code>	<code>Admin</code> , <code>SuperAdmin</code> , <code>soporte_tecnico</code>

Constante	Valor
RegisterableRoles	Club,Admin,Largador,Cronometrista,JuezControl,ControlTecnico,soporte_tecnico (excluye SuperAdmin)
PrivilegedRoles	Admin,SuperAdmin,soporte_tecnico (evita degradar admins a Club)

8.2 Controles implementados

- **JWT** HMAC-SHA512; TokenKeyResolver (fallback en Development, obligatorio fuera).
- **Enforcement** en LoginAsync / GetMeAsync (bloquea inactivo/bloqueado por pago/vencido, o X-Client-App sin Acceso*) y RegisterAsync (CanCreateRole).
- **Lockout**: 5 intentos → EstaActivo = false .
- **Rate limiting** (auth 20/min, live 120/min), **security headers**, **CORS** whitelist, **FileUploadRules** (6 MB), **errores sanitizados**.
- **Aislamiento de logins por federación**: TenantScopeHelper , TenantProvider , AuthService.GetUsuariosAsync , AuditoriaController.BuildScopedQuery .

8.3 Endpoints anónimos (revisados)

POST api/Auth/login , POST api/Auth/logout , POST api/Auth/solicitar-reset-password , GET api/Eventos/proximos , GET api/Eventos/{id} , GET api/Eventos/{id}/fases , GET api/Eventos/{id}/pruebas , GET api/Resultados/Fase/{id} , GET api/Fases/EventoPrueba/{id} , GET api/Fases/all-by-evento/{id} , GET api/SaaS/planes , api/Health* , POST api/Notificacion/webhook , POST api/Support/frontend-error .

9. SignalR

- **Hub**: /hubs/timing (MapHub<TimingHub>().AllowAnonymous()).
- **Grupos**: race_{faseId} , event_{eventoId} , operators , user_{username} , fed_{federacionId} .
- **Presencia**: RacePresenceUpdated , EventPresenceUpdated (diccionarios en memoria).
- **Métodos**: ver §3.11 del Manual de Uso API.
- **Eventos**: RaceStarted , RaceFinished , RaceReset , RaceInReview , globalRaceStarted , globalRaceOfficialized , globalRaceInReview , GlobalResultStatusUpdated , TimeReceived , globalTimeReceived , LapRecorded , ResultadoActualizado , RacePresenceUpdated , EventPresenceUpdated , newMessageReceived , newEventCreated , paymentStatusChangeRequested .

10. Observabilidad y auditoría

- **Auditoría** en tabla public.Auditoria : acción, usuario, detalle, timestamp UTC. Acciones destacadas: GENERATE_HEATS_AUTO , GENERATE_LARGADA_MARATON , PROMOTE_STAGE , RESET_RACE , SAVE_TIMING , ERROR_FATAL .
- **Logs de soporte** vía api/Support/logs ; errores de frontend vía POST api/Support/frontend-error .
- **Diagnóstico** vía api/Diagnostic/* .
- **Audiencia** (AudiencePresenceTracker + snapshots) con capacidad configurable (AudienceMonitoring:SoftCapacity , default 200).

11. Backups

BackupService (SuperAdmin/soporte):

- scope=full → pg_dump con PGPASSWORD por variable de entorno.
- scope=federacion&idFederacion=N → generación de INSERTs de esa federación.
- Historial leído desde Auditoría (GET api/Backup/history).
- En Docker se instala postgresql-client-18 (pg_dump ≥ server; Render PG 18.x).

12. Docker y despliegue

Dockerfile multi-stage:

Etap	Imagen	Acción
build	mcr.microsoft.com/dotnet/sdk:10.0-noble	dotnet restore + build -c Release
publish	(build)	dotnet publish ... /p:UseAppHost=false
final	mcr.microsoft.com/dotnet/aspnet:10.0-noble	Instala postgresql-client-18 (+ symlinks pg_dump / pg_restore / psql), copia publish

Variables en imagen: ASPNETCORE_URLS=http://+:8080 , EXPOSE 8080 , DOTNET_HOSTBUILDER__RELOADCONFIGONCHANGE=false , DOTNET_USE_POLLING_FILE_WATCHER=true . Entrypoint: dotnet SportTrack-Sigdef.dll .

Render: push a main → auto-deploy. Sin CI ni docker-compose en el repo. Ver *Manual de Instalación y Despliegue*.

13. Deuda técnica y discrepancias conocidas

#	Deuda / discrepancia	Detalle	Impacto
D-01	MaxAtletas no aplicado	AltaAtletaService no valida el tope del plan (contradice Diseno-Enforcement-Planes \$5.3)	El plan no limita atletas reales
D-02	MaxTorneosActivos sin efecto	Migración RemovePlanTournamentLimits lo deja en -1 ; no hay enforcement	Sin límite real de torneos
D-03	Cookie-first documentado como "fuera de alcance"	Ya está implementado (X-Access-Token + Authorization + ? access_token)	Doc histórica desactualizada
D-04	Auditoria.IdEvento [NotMapped] + filtro inconsistente	AuditoriaController.cs:47 filtra a.IdEvento == eventoId.Value sobre una propiedad no mapeada; scripts/backfill-auditoria-id-evento.sql asume una columna inexistente	Riesgo: la consulta no refleja la intención o falla según traducción EF
D-05	Webhook MercadoPago sin validación de firma	POST api/Notificacion/webhook es anónimo y no valida firma	Riesgo de falsificación de notificaciones
D-06	WeatherForecastController legacy	Controlador plantilla autenticado sin uso de negocio	Ruido; conviene eliminarlo
D-07	Enums duplicados	Alias como CategoriaEdad / CategoriaEdadEnum	Confusión de mantenimiento
D-08	Doc desactualizada (.NET 8 / puerto 5029)	docs/README.md , guias/operacion-local.md	Onboarding incorrecto

14. Migraciones y seed

14.1 Migraciones (orden cronológico)

InitialCreate , UniqueParticipanteDocumento , FixUtf8PlanNames , AddMensajeria , AddCampanasEnvio , AddEntrenadorLicencia , AddSistemaOrigenMensajeria , PlanSaaSEnforcementFlags , AddTraspasosAtletas , AddGapRecuperacionMinutos , AddEventoModalidad , AddGrupoLargadaId , AddDistancia6000m , AddAudiencePeakSnapshots , AddAudienceMonitorSettings , RemovePlanTournamentLimits , AddPlanPrecioAnual , AddPlanDescuentoAnual , SolicitudTraspasoOrigenNullable , AddTimingSubmissionOutbox , AddPermitirMezclarCategorias (última, 2026-09-16).

Aplicación: automática al arranque (Database.MigrateAsync si hay pendientes) o manual dotnet ef database update .

14.2 Seed

Catálogo	Registros
Sexos	3
Botes	6
Categorías	11
Distancias	17 (incluye 6000 m, id 17)
PlanesSaaS	9 (SIGDEF S/M/L = 1-3, SportTrack S/M/L = 4-6, Pack Dúo S/M/L = 7-9)
Usuario	admin / admin123 (SuperAdmin, BCrypt) — cambiar en producción

15. Estrategia offline / outbox

- El cronometrista opera online por SignalR; si pierde conexión, **encola** envíos en public.TimingSubmissionOutbox (POST api/timing-outbox), con upsert por (FaseId , Username) y TTL 24 h.
- CommitAsync aplica los tiempos como BatchUpdate y dispara FinalizarFase o EnviarARrevision .
- FlushPendingAsync reintenta; PurgeExpiredAsync limpia expirados.
- Soporte puede forzar el commit / descartar entradas vía api/Support/timing-outbox/** .
- No existe token de cronometrista separado**; se reutiliza el JWT con SessionLifetimePolicy extendida.

16. Testing y mantenimiento

- **No hay proyecto de tests** en la solución (deuda de cobertura).
 - Verificación manual recomendada: Swagger en Development, `/api/Health` y `/api/Health/db`, y los diagramas de `docs/tecnico/`.
 - **Checklist de mantenimiento:**
 1. Mantener `net10.0` y Npgsql alineados.
 2. Aplicar migraciones nuevas con `dotnet ef migrations add`.
 3. Revisar la deuda de §13 (especialmente D-01, D-04, D-05).
 4. Verificar CORS si se agregan dominios de frontend.
 5. Renovar `TokenKey` y credenciales `admin` en producción.
-

Pie de página

-  [06 · Manual de Instalación y Despliegue](#)
-  [04 · Manual de Uso de la API](#)
-  [01 · Requerimientos · Volver al índice del Pack](#)

06 · Manual de Instalación y Despliegue

Última actualización: 2026-09-27 Audiencia: desarrolladores, DevOps y responsables de despliegue.

Índice

- 1. Requisitos
- 2. Obtención del código
- 3. Configuración (appsettings / variables)
- 4. Ejecución local
- 5. Migraciones
- 6. Build y publicación
- 7. Docker
- 8. Despliegue en Render
- 9. Verificación
- 10. Checklist de despliegue
- 11. Troubleshooting

1. Requisitos

Requisito	Versión / notas
.NET SDK	10.0 (target net10.0)
PostgreSQL	14+ (Render usa PG 18.x)
pg_dump	Debe ser ≥ versión del servidor (para backups)
Docker	Opcional (imagen multi-stage)
EF Core CLI	dotnet tool install --global dotnet-ef (para migraciones manuales)
Git	Para clonar el repositorio

2. Obtención del código

```
git clone <url-del-repositorio> SportTrack-Sigdef
cd SportTrack-Sigdef
dotnet restore SportTrack-Sigdef.sln
```

La solución SportTrack-Sigdef.sln contiene 4 proyectos: SportTrack-Sigdef (Api), SportTrack-Sigdef.Controladores , SportTrack-Sigdef.AccesoDatos y SportTrack-Sigdef.Entidades .

3. Configuración (appsettings / variables)

3.1 appsettings.json (base)

```
{
  "Logging": { "LogLevel": { "Default": "Information", "Microsoft.AspNetCore": "Warning" } },
  "AllowedHosts": "*",
  "AudienceMonitoring": { "SoftCapacity": 200 },
  "CloudinarySettings": { "CloudName": "", "ApiKey": "", "ApiSecret": "" }
}
```

3.2 Variables de entorno

Variable	Obligatoria	Descripción
ConnectionStrings__DefaultConnection	Sí (o DATABASE_URL)	Cadena Npgsql de PostgreSQL
DATABASE_URL	Alternativa	Render la provee; acepta postgres:// (se normaliza a postgresql://)

Variable	Obligatoria	Descripción
TokenKey	Sí en producción	Clave de firma JWT (en Development hay fallback)
AllowedOrigins	No	Lista CSV adicional de orígenes CORS
CloudinarySettings__CloudName	No	Cloudinary (o CLOUDINARY_CLOUD_NAME)
CloudinarySettings__ApiKey	No	Cloudinary (o CLOUDINARY_API_KEY)
CloudinarySettings__ApiSecret	No	Cloudinary (o CLOUDINARY_API_SECRET)
AudienceMonitoring__SoftCapacity	No	Capacidad de audiencia (default 200)
ASPNETCORE_ENVIRONMENT	No	Development habilita Swagger; en producción dejarlo sin Swagger
PGPASSWORD	Para backups	Contraseña de PostgreSQL para pg_dump

 **TokenKey es obligatorio fuera de Development.** Si falta, el arranque debe fallar/no emitir tokens válidos.

3.3 launchSettings.json (perfiles locales)

Perfil	URL
http	http://localhost:5012
https	https://localhost:7232;http://localhost:5012
IIS Express	http://localhost:10202 / https://localhost:44390

Corrección: documentos previos indicaban el puerto **5029**; el puerto real es **5012**.

4. Ejecución local

```
cd SportTrack-Sigdef
dotnet run
```

- Con el perfil por defecto: API en http://localhost:5012 .
- Swagger (solo Development): http://localhost:5012/swagger .
- Health: http://localhost:5012/api/Health y http://localhost:5012/api/Health/db .
- Hub SignalR: http://localhost:5012/hubs/timing .

Al arrancar, el servicio:

1. Verifica conexión a PostgreSQL.
2. Aplica migraciones pendientes (Database.MigrateAsync).
3. Ejecuta EventoEstadoSyncService.SyncAllAsync .
4. Levanta el background service de estados (cada 15 min) y el de snapshots de audiencia.

5. Migraciones

5.1 Automáticas (recomendado)

Se aplican solas al iniciar si hay pendientes. No requiere intervención.

5.2 Manuales

```
dotnet ef migrations add <NombreMigracion> --project SportTrack-Sigdef.AccesoDatos --startup-project SportTrack-Sigdef
dotnet ef database update --project SportTrack-Sigdef.AccesoDatos --startup-project SportTrack-Sigdef
```

Última migración: AddPermitirMezclarCategorias (2026-09-16).

6. Build y publicación

```
dotnet build SportTrack-Sigdef.sln -c Release
dotnet publish SportTrack-Sigdef/SportTrack-Sigdef.csproj -c Release -o ./publish /p:UseAppHost=false
```

7. Docker

7.1 Construir y ejecutar

```
docker build -t sporttrack-sigdef .
docker run -p 8080:8080 \
  -e ASPNETCORE_ENVIRONMENT=Production \
  -e ConnectionStrings__DefaultConnection="Host=...;Database=...;Username=...;Password=..." \
  -e TokenKey="<clave-larga-y-segura>" \
  sporttrack-sigdef
```

7.2 Características de la imagen

Aspecto	Detalle
Build	mcr.microsoft.com/dotnet/sdk:10.0-noble
Runtime	mcr.microsoft.com/dotnet/aspnet:10.0-noble
PostgreSQL client	postgresql-client-18 + symlinks a /usr/local/bin
Puerto	ASPNETCORE_URLS=http://+:8080, EXPOSE 8080
Watchers	DOTNET_HOSTBUILDER__RELOADCONFIGONCHANGE=false, DOTNET_USE_POLLING_FILE_WATCHER=true (fix Render free tier)

No hay docker-compose ni pipeline CI en el repositorio.

8. Despliegue en Render

1. **Repositorio conectado a Render** con auto-deploy en push a `main`.
2. Render construye con el `Dockerfile` (build + publish + runtime).
3. Configurar **variables de entorno** en el panel de Render:

Variable	Valor
ASPNETCORE_ENVIRONMENT	Production
TokenKey	Clave segura y larga
ConnectionStrings__DefaultConnection	(o dejar que use DATABASE_URL de Render)
AllowedOrigins	CSVs de dominios adicionales si aplica
CloudinarySettings__CloudName / __ApiKey / __ApiSecret	Credenciales Cloudinary
AudienceMonitoring__SoftCapacity	(opcional)
PGPASSWORD	Contraseña PG para pg_dump

4. El servicio escucha en `8080` (Render lo enruta por HTTPS).
5. `UseForwardedHeaders` lee la IP/esquema real del proxy.
6. Las migraciones se aplican al primer arranque tras el deploy.

9. Verificación

Chequeo	Cómo
Servicio arriba	GET /api/Health → 200
Base de datos	GET /api/Health/db → 200
Migraciones al día	Logs de arranque: "Base de datos al día"
Login	POST /api/Auth/login con admin/admin123 (cambiar en prod)
Live público	GET /api/Eventos/proximos sin token → 200
Auth por defecto	GET /api/Auth/me sin token → 401
CORS	Probar desde el dominio del frontend con credenciales
Backups	GET /api/Backup/download?scope=full con SuperAdmin
SignalR	Conectar a /hubs/timing y llamar GetServerTime

10. Checklist de despliegue

- ☐ `TokenKey` configurado (obligatorio en prod).
- ☐ Cadena de conexión / `DATABASE_URL` configurada.
- ☐ `ASPNETCORE_ENVIRONMENT=Production` (Swagger deshabilitado).
- ☐ `AllowedOrigins` con los dominios del frontend en producción.
- ☐ Credenciales Cloudinary configuradas (subida de documentación).
- ☐ `PGPASSWORD` configurado (backups).
- ☐ Imagen Docker con `postgresql-client-18`.
- ☐ Verificar `/api/Health` y `/api/Health/db` tras el deploy.
- ☐ Cambiar credenciales seed `admin/admin123`.
- ☐ Confirmar migraciones aplicadas en los logs.

11. Troubleshooting

Síntoma	Causa probable	Solución
ERROR: No se pudo conectar a PostgreSQL	Cadena de conexión mal formada o ausente	Revisar <code>ConnectionStrings__DefaultConnection</code> o <code>DATABASE_URL</code> (formato <code>postgresql://</code>)
Arranque falla por TokenKey	Falta <code>TokenKey</code> en producción	Definir <code>TokenKey</code> en variables de entorno
401 en todas las llamadas	Falta token válido / usuario inactivo / plan vencido	Revisar login y estado del usuario
403 en Live o consolas de juez	Plan sin <code>AccesoControlesLive</code> / origen sin acceso	Verificar plan de la federación (IDs 6 y 9)
429 frecuente	Rate limit <code>auth</code> (20/min) o <code>live</code> (120/min) por IP	Reducir frecuencia; revisar IP detrás del proxy
CORS bloqueado	Dominio no incluido en la whitelist	Agregar el dominio a <code>AllowedOrigins</code>
Falla el backup	<code>pg_dump</code> ausente o versión menor al server	Verificar <code>postgresql-client-18</code> en la imagen
Error de SSL local	Certificado HTTPS de desarrollo	<code>dotnet dev-certs https --trust</code>
Swagger no aparece	<code>ASPNETCORE_ENVIRONMENT</code> distinto de <code>Development</code>	Usar el perfil <code>http</code> / <code>https</code> local
Migraciones no aplican	Error en el arranque silenciado en log	Revisar logs de arranque; aplicar manual con <code>dotnet ef database update</code>

Pie de página

- [➡ 05 · Manual Técnico](#)
- [➡ 04 · Manual de Uso de la API](#)
- [➡ Volver al índice del Pack · Documentación histórica](#)